



Capturing Types

ALEKSANDER BORUCH-GRUSZECKI and MARTIN ODERSKY, EPFL
EDWARD LEE and ONDŘEJ LHOTÁK, University of Waterloo
JONATHAN BRACHTHÄUSER, Eberhard Karls University of Tübingen

21

Type systems usually characterize the shape of values but not their free variables. However, many desirable safety properties could be guaranteed if one knew the free variables captured by values. We describe $CC_{<:\square}$, a calculus where such captured variables are succinctly represented in types, and show it can be used to safely implement effects and effect polymorphism via scoped capabilities. We discuss how the decision to track captured variables guides key aspects of the calculus, and show that $CC_{<:\square}$ admits simple and intuitive types for common data structures and their typical usage patterns. We demonstrate how these ideas can be used to guide the implementation of capture checking in a practical programming language.

CCS Concepts: • **Theory of computation** → **Type structures**; • **Software and its engineering** → *Object oriented languages*;

Additional Key Words and Phrases: Scala, type systems, effects, resources, capabilities

ACM Reference format:

Aleksander Boruch-Gruszecki, Martin Odersky, Edward Lee, Ondřej Lhoták, and Jonathan Brachthäuser. 2023. Capturing Types. *ACM Trans. Program. Lang. Syst.* 45, 4, Article 21 (November 2023), 52 pages. <https://doi.org/10.1145/3618003>

1 INTRODUCTION

Effects are aspects of computation that go beyond describing shapes of values and that we still want to track in types. What exactly is modeled as an effect is a question of language or library design. Some possibilities are *reading* or *writing* to mutable state outside a function; *throwing an exception* to signal abnormal termination of a function; *I/O* including file operations, network access, or user interaction; *non-terminating* computations; *suspending* a computation, such as waiting for an event; or *using a continuation* for control operations.

Despite hundreds of published papers, there is comparatively little adoption of static effect checking in programming languages. The few designs that *are* widely implemented (e.g., Java’s checked exceptions or monadic effects in some functional languages) are often critiqued for being both too verbose and too rigid. The problem is not lack of expressiveness—systems have been proposed and implemented for many kinds of effects. Rather, the problem is the lack of usability

A. Boruch-Gruszecki and M. Odersky contributed equally to this article.

This research was partially funded by the Natural Sciences and Engineering Research Council of Canada.

Authors’ addresses: A. Boruch-Gruszecki and M. Odersky, EPFL; emails: {aleksander.boruch-gruszecki, martin.odersky}@epfl.ch; E. Lee and O. Lhoták, University of Waterloo; emails: {e45lee, olhotak}@uwaterloo.ca; J. Brachthäuser, Eberhard Karls University of Tübingen; email: jonathan.brachthaeuser@unituebingen.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0164-0925/2023/11-ART21 \$15.00

<https://doi.org/10.1145/3618003>

and flexibility, with particular difficulties in describing polymorphism. This leads either to overly complex definitions or to the requirement to duplicate large bodies of code.

Classical type-systematic approaches fail since effects are inherently transitive along the edges of the dynamic call graph: a function’s effects include the effects of all the functions it calls, transitively. Traditional type and effect systems have no lightweight mechanism to describe this behavior. The standard approach is either manual specialization along specific effect classes, which means large-scale code duplication, or quantifiers on all definitions along possible call graph edges to account for the possibility that some call target has an effect, which means large amounts of boilerplate code. Arguably, it is this problem more than any other that has so far hindered wide-scale application of effect systems.

A promising alternative that circumvents this problem is to model effects via capabilities tracked in the type system [Brachthäuser et al. 2020a; Craig et al. 2018; Gordon 2020; Liu 2016; Marino and Millstein 2009; Miller 2006; Osvald et al. 2016]. Capabilities exist in many forms, but we will restrict the meaning here to simple object capabilities represented as regular program variables. For instance, consider the following two morally equivalent formulations of a method in Scala:¹

```
def f(): T throws E
def f()(using ct: CanThrow[E]): T
```

The first version looks like it describes an effect: function *f* returns a *T*, or it might throw exception *E*. The effect is mentioned in the return type `throws[T, E]` where the `throws` is written infix.

The second version expresses analogous information as a capability: function *f* returns a value of type *T*, *provided* it can be passed a capability *ct* of type `CanThrow[T]`. The capability is modeled as a parameter. To avoid boilerplate, that parameter is synthesized automatically by the compiler at the call site assuming a matching capability is defined there. This is expressed by a `using` keyword, which indicates that a parameter is implicit in Scala 3 (Scala 2 would have used the `implicit` keyword instead). The fact that capabilities are implicit rather than explicit parameters helps with conciseness and readability of programs but is not essential for understanding the concepts discussed in this work.

Aside. The link between the “effect” and the “capability” version of *f* can be made more precise by means of context function types [Odersky et al. 2018]. It is embodied in the following definition of the `throws` type:

```
infix type throws[T, E <: Exception] = CanThrow[E] => T
```

The context function type `CanThrow[E] => T` represents functions from `CanThrow[E]` to *T* that are applied implicitly to arguments synthesized by the compiler. This gives a direct connection between the effect view based on the `throws` type and the capability view based on its expansion.

An important benefit of this switch from effects to capabilities is that it gives us polymorphism for free. For instance, consider the `map` function in class `List[A]`. If we wanted to take effects into account, it would look like this:

```
def map[B, E](f: A -> B eff E): List[B] eff E
```

Here, `A -> B eff E` is hypothetical syntax for the type of functions from *A* to *B* that can have effect *E*. While looking reasonable in the small, this scheme quickly becomes unmanageable, if we consider that every higher-order function has to be expanded that way and, furthermore, that in an object-oriented language almost every method is a higher-order function [Cook 2009]. Indeed,

¹Scala 3.1 with language import `saferExceptions` enabled.

```

class TooLarge extends Exception

def f(x: Int): Int throws TooLarge =
  if x < limit then x * x
  else throw TooLarge()

val xs: List[Int]
try xs.map(x => f(x))
catch case TooLarge => Nil

def f(x: Int)
  (using CanThrow[TooLarge]): Int =
  if x < limit then x * x
  else throw TooLarge()

val xs: List[Int]
try xs.map(x =>
  f(x)(using new CanThrow[TooLarge]))
catch case TooLarge => Nil

```

Fig. 1. Exception handling: source (left) and compiler-generated code (right).

many designers of programming languages with support for effect systems agree that programmers should ideally not be confronted with explicit effect quantifiers [Brachthäuser et al. 2020a; Leijen 2017; Lindley et al. 2017].

However, here is the type of `map` if we represent effects with capabilities.

```
def map(f: A => B): List[B]
```

Interestingly, this is exactly the same as the type of `map` in current Scala, which does not track effects! In fact, compared to effect systems, we now decompose the space of possible effects differently: `map` is classified as pure since it does not produce any effects in its own code, but when analyzing an application of `map` to some argument, the capabilities required by the argument are also capabilities required by the whole expression. In that sense, we get effect polymorphism for free.

The reason this works is that in an effects-as-capabilities discipline, the type $A \Rightarrow B$ represents the type of *impure* function values that can *close over* arbitrary effect capabilities. Alongside, we also define a type of pure functions $A \rightarrow B$ that are not allowed to close over capabilities.

This seems almost too good to be true, and indeed there is a catch: it now becomes necessary to reason about capabilities captured in closures.

To see why, consider that effect capabilities are often scoped and therefore have a limited lifetime. For instance a `CanThrow[E]` capability would be generated by a `try` expression that catches `E`. It is valid only as long as the `try` is executing. Figure 1 shows an example of capabilities for checked exceptions, both as source syntax on the left and with compiler-generated implicit capability arguments on the right. The following slight variation of this program would throw an unhandled exception since the function `f` is now evaluated only when the iterator's `next` method is called, which is after the `try` handling the exception has exited.

```

val it =
  try xs.iterator.map(f)
  catch case TooLarge => Iterator.empty
it.next()

```

A question answered in this article is how to rule out `Iterator`'s lazy `map` statically while still allowing `List`'s strict `map`. A large body of research exists that could address this problem by restricting reference access patterns. Relevant techniques include linear types [Wadler 1990], rank 2 quantification [Launchbury and Sabry 1997], regions [Grossman et al. 2002; Tofte and Talpin 1997], uniqueness types [Barendsen and Smetsers 1996], ownership types [Clarke et al. 1998; Noble et al. 1998], and second-class values [Osvald et al. 2016]. A possible issue with many of these approaches is their relatively high notational overhead, particularly when dealing with polymorphism.

The approach we pursue here is different. Instead of restricting certain access patterns *a priori*, we focus on describing what capabilities are possibly captured by values of a type. At its core there are the following two interlinked concepts:

- A *capturing type* is of the form $T^{c_1, \dots, c_n}$ where T is a type and $\{c_1, \dots, c_n\}$ is a *capture set* of capabilities.
- A *capability* is a parameter or local variable that has as type a capturing type with non-empty capture set. We call such capabilities *tracked* variables.

Every capability gets its authority from some other, more sweeping capabilities which it captures. The most sweeping capability, from which ultimately all others are derived, is “cap”, the *universal capability*.

As an example how capabilities are defined and used, consider a typical *try-with-resources* pattern:

```
def usingFile[T](name: String, op: OutputStream^{cap} => T): T =
  val f = new FileOutputStream(name)
  val result = op(f)
  f.close()
  result

val xs: List[Int] = ...
def good = usingFile("out", f => xs.foreach(x => f.write(x)))
def fail =
  val later = usingFile("out",
    f => (y: Int) => xs.foreach(x => f.write(x + y)))
  later(1)
```

The usingFile method runs a given operation op on a freshly created file, closes the file, and returns the operation’s result. The method enables an effect (writing to a file) and limits its validity (to until the file is closed). Function good invokes usingFile with an operation that writes each element of a given list xs to the file. By contrast, function fail represents an illegal usage: it invokes usingFile with an operation that returns a function that, when invoked, will write list elements to the file. The problem is that the writing happens in the application later(1) when the file has already been closed.

We can accept the first usage and reject the second by marking the output stream passed to op as a capability. This is done by adding the capture set annotation cap after the type proper. Our implementation of capture checking rejects the second usage with an error message that the result of usingFile leaks f.

This example used a capability parameter that was directly derived from cap. But capabilities can also be derived from other non-universal capabilities. For instance:

```
def usingLogFile[T](f: OutputStream^{cap}, op: Logger^{f} => T): T =
  op(new Logger(f))
```

The usingLogFile method takes an output stream (which is a capability) and an operation, which gets passed a Logger. The Logger capability is derived from the output stream capability, as can be seen from its type $Logger^f$.

This article develops a *capture calculus*, $CC_{< \Box}$, as a foundational type system that allows reasoning about scoped capabilities. By sketching a prototype language design based on this calculus, we argue that it is expressive enough to support a wide range of usage patterns with very low notational overhead. The article makes the following specific contributions:

- We define a simple yet expressive type system for tracking captured capabilities in types. The calculus extends System $F_{<}$ with capture sets of capabilities.

- We prove progress and preservation of types relative to a small-step evaluation semantics. We also prove a capture prediction lemma that states that capture sets in types over-approximate captured variables in runtime values.
- We illustrate the practical applicability of the calculus with a number of examples that have been checked by a prototype capture checker implemented within the Scala 3 compiler.

The presented design is at the same time simple in theory and concise and flexible in its practical application. We demonstrate that the following elements are essential for achieving good usability:

- Use reference-dependent typing, where a formal function parameter stands for the potential references captured by its argument [Brachthäuser et al. 2022; Odersky et al. 2021]. This avoids the need to introduce separate binders for capabilities or effects. Technically, this means that references (but not general terms) can form part of types as members of capture sets. A similar approach is taken in the path-dependent typing discipline of DOT [Amin et al. 2016; Rompf and Amin 2016] and by reachability types for alias checking [Bao et al. 2021].
- Employ a subtyping discipline that mirrors subsetting of capabilities and that allows capabilities to be refined or abstracted. Subtyping of capturing types relies on a new notion of *subcapturing* that encompasses both subsetting (smaller capability sets are more specific than larger ones) and derivation (a capability singleton set is more specific than the capture set of the capability's type). Both dimensions are essential for a flexible modeling of capability domains.
- Limit propagation of capabilities in instances of generic types where they cannot be accessed directly. This is achieved by boxing types when they enter a generic context and unboxing on every use site [Brachthäuser et al. 2022].

Whereas many of our motivating examples describe applications in effect checking, the formal treatment presented here does not mention effects. In fact, the effect domains are intentionally kept open since they are orthogonal to the aim of the article. Effects could be exceptions, file operations, or region allocations, but also algebraic effects, IO, or any sort of monadic effects. To express more advanced control effects, one usually needs to add continuations to the operational semantics, or use an implicit translation to the continuation monad. In short, capabilities can delimit what effects can be performed at any point in the program, but they by themselves do not perform an effect [Brachthäuser et al. 2020a; Gordon 2020; Liu 2016; Marino and Millstein 2009; Osvald et al. 2016]. For that, one needs a library or a runtime system that would be added as an extension of $CC_{<:\Box}$. Since $CC_{<:\Box}$ is intended to work with all such effect extensions, we refrain from adding a specific application to its core operational semantics.

We introduce later an extension of $CC_{<:\Box}$ to demonstrate that scoping is properly enforced. The extension adds just enough primitives to $CC_{<:\Box}$ so that ill-scoped programs could “go wrong” at runtime, then proceeds to show that all such programs are ruled out by the type system.

The version of $CC_{<:\Box}$ presented here evolved from a system that was originally proposed to make exception checking safe [Odersky et al. 2021]. The earlier paper described a way to encode information about potentially raised exceptions as object capabilities passed in parameters. It noted that the proposed system is not completely safe since capabilities can escape in closures, and it hypothesized a possible way to fix the problem by presenting a draft of what became $CC_{<:\Box}$. At the time, the metatheory of the proposed system was not worked out yet and the progress and preservation properties were left as conjectures. This work presents a fully worked-out metatheory with proofs of type soundness as well as a semantic characterization of capabilities. There are some minor differences in the operational semantics, which were necessary to make the progress theorem go through. We also present a range of use cases outside of exception handling, demonstrating the broad applicability of the calculus.

The rest of this article is organized as follows. Section 2 explains and motivates the core elements of our calculus. Section 3 presents $CC_{< \square}$. Section 4 lays out its metatheory. Section 5 illustrates the expressiveness of typing disciplines based on the calculus in examples. Section 6 illustrates the role of the $CC_{< \square}$'s boxing feature in making polymorphism sound. Section 7 presents an extension of $CC_{< \square}$ for demonstrating that scoping is enforced. Section 8 discusses related work, and Section 9 concludes.

2 INFORMAL DISCUSSION

This section motivates and discusses some of the key aspects of capture checking. All examples are written in an experimental language extension of Scala 3 [Scala 2022b] and were compiled with our prototype implementation of a capture checker [Scala 2022c].

2.1 Capability Hierarchy

We saw in Section 1 that every capability except `cap` is created from some other capabilities which it retains in the capture set of its type. Here is an example that demonstrates this principle:

```
class FileSystem

class Logger(fs: FileSystem^{cap}):
  def log(s: String): Unit = ... // Write to a log file, using `fs`

def test(fs: FileSystem^{cap}): LazyList[Int]^{fs} =
  val lgr: Logger^{fs} = new Logger(fs)
  lgr.log("hello_world!")
  val xs: LazyList[Int]^{lgr} =
    LazyList.from(1)
      .map { i =>
        lgr.log(s"computing_elem_#_$i")
        i * i
      }
  xs
```

Here, the `test` method takes a `FileSystem` as a parameter. `fs` is a capability since its type has a non-empty capture set. The capability is passed to the `Logger` constructor and retained as a field in class `Logger`. Hence, the local variable `lgr` has type `Logger^{fs}`: it is a `Logger` that retains the `fs` capability.

The second variable defined in `test` is `xs`, a lazy list that is obtained from `LazyList.from(1)` by logging and mapping consecutive numbers. Since the list is lazy, it needs to retain the reference to the logger `lgr` for its computations. Hence, the type of the list is `LazyList[Int]^{lgr}`. However, since `xs` only logs but does not do other file operations, it retains the `fs` capability only indirectly. That is why `fs` does not show up in the capture set of `xs`.

Capturing types come with a subtype relation where types with “smaller” capture sets are subtypes of types with larger sets (the *subcapturing* relation is defined in more detail in the following). If a type `T` does not have a capture set, it is called *pure* and is a subtype of any capturing type that adds a capture set to `T`.

2.2 Function Types

The function type `A => B` stands for a function that can capture arbitrary capabilities. We call such functions *impure*. By contrast, the new single arrow function type `A -> B` stands for a function that

cannot capture any capabilities, or otherwise said, is *pure*. One can add a capture set after the arrow of an otherwise pure function. For instance, $A \rightarrow \{c, d\} B$ would be a function that can capture capabilities c and d , but no others. It can be seen as a shorthand for the type $(A \rightarrow B)^{\{c, d\}}$.

The impure function type $A \Rightarrow B$ is treated as an alias for $A \rightarrow \{\text{cap}\} B$. In other words, impure functions are functions that can capture anything.

Note. Like other object-functional languages, Scala distinguishes between functions and methods (which are defined using **def**). Functions are values, whereas methods represent pieces of code that logically form part of the enclosing object. The type system treats method signatures and function types separately: a function type is treated as an object type with a single apply method. Methods are converted to functions by eta expansion—that is, the unapplied method reference m is transparently converted to the function value $x \Rightarrow m(x)$.

Since methods are not values in Scala, they never capture anything directly. Therefore, the distinctions between pure vs impure function types do not apply to methods. The capabilities captured by a method would show up in the object closure of which the method forms part.

2.3 Capture Checking of Closures

If a closure refers to capabilities in its body, it captures these capabilities in its type. For instance, consider the following:

```
def test(fs: FileSystem): String -> {fs} Unit =
  (x: String) => Logger(fs).log(x)
```

Here, the body of `test` is a lambda that refers to the capability `fs`, which means that `fs` is retained in the lambda. Consequently, the type of the lambda is $\text{String} \rightarrow \{fs\} \text{Unit}$.

Note. On the term level, function values are always written with $\Rightarrow$ (or $\Rightarrow?$ for context functions). There is no syntactic distinction between pure and impure function values. The distinction is only made in their types.

A closure also captures all capabilities that are captured by the functions it calls. For instance, in

```
def test(fs: FileSystem) =
  val f = (x: String) => Logger(fs).log(x)
  val g = (x: String) => f(x)
  g
```

the result of `test` has type $\text{String} \rightarrow \{fs\} \text{Unit}$ even though function `g` itself does not refer to `fs`.

2.4 Subtyping and Subcapturing

Capturing influences subtyping. As usual, we write $T_1 <: T_2$ to express that the type T_1 is a subtype of the type T_2 , or, equivalently, that T_1 conforms to T_2 . An analogous *subcapturing* relation applies to capture sets. If C_1 and C_2 are capture sets, we write $C_1 <: C_2$ to express that C_1 is *covered by* C_2 , or, swapping the operands, that C_2 *covers* C_1 .

Subtyping extends as follows to capturing types:

- Pure types are subtypes of capturing types. In other words, $T <: T \wedge C$, for any type T and capturing set C .
- For capturing types, smaller capture sets produce subtypes: $T_1 \wedge C_1 <: T_2 \wedge C_2$ if $C_1 <: C_2$ and $T_1 <: T_2$.

A subcapturing relation $C_1 <: C_2$ holds if C_2 *accounts for* every element c in C_1 . This means one of the following two conditions must be true:

- $c \in C_2$,
- c 's type has capturing set C and C_2 accounts for every element of C (i.e., $C <: C_2$).

Example. Given

```
fs: FileSystem^{cap}
ct: CanThrow[Exception]^{cap}
l : Logger^{fs}
```

we have

```
{l} <: {fs}      <: {cap}
{fs} <: {fs, ct} <: {cap}
{ct} <: {fs, ct} <: {cap}
```

The set consisting of the root capability $\{cap\}$ covers every other capture set. This is a consequence of the fact that, ultimately, every capability is created from cap .

2.5 Capture Tunneling

Next, we discuss how type polymorphism interacts with reasoning about capture. To this end, consider the following simple definition of a `Pair` class:²

```
class Pair[+A, +B](x: A, y: B):
  def fst: A = x
  def snd: B = y
```

What happens if we pass arguments to the constructor of `Pair` that capture capabilities?

```
def x: Int ->{ct} String
def y: Logger^{fs}
def p = Pair(x, y)
```

Here the arguments x and y close over different capabilities ct and fs , which are assumed to be in scope. So what should be the type of p ? Maybe surprisingly, it will be typed as follows:

```
def p: Pair[Int ->{ct} String, Logger^{fs}] = Pair(x, y)
```

In other words, the outer capture set is empty and it neither mentions ct nor fs , even though the value `Pair(x, y)` *does* capture them. So why do they not show up in its type at the outside?

Although assigning p the capture set $\{ct, fs\}$ would be sound, types would quickly grow inaccurate and unbearably verbose. To remedy this, $CC_{< \square}$ performs *capture tunneling*. Once a type variable is instantiated to a capturing type, the capture is not propagated beyond this point. However, if the type variable is instantiated again on access, the capture information “pops out” again.

Even though p is technically untracked because its capture set is empty, writing $p.fst$ would record a reference to the captured capability ct . So if this access was put in a closure, the capability would again form part of the outer capture set. For example,

```
() => p.fst : () ->{ct} Int ->{ct} String
```

In other words, references to capabilities “tunnel through” generic instantiations—from creation to access; they do not affect the capture set of the enclosing generic data constructor applications.

As mentioned previously, this principle plays an important part in making capture checking concise and practical. To illustrate, let us take a look at the following example:

²This class is covariant in both A and B , as denoted by the pluses.


```
def mapFirst[A,B,C](p: Pair[A,B], f: A => C): Pair[C,B] =
  Pair(f(p.x), p.y)
```

Relying on capture tunneling, neither the types of the parameters to `mapFirst` nor its result type need to be annotated with capture sets. Intuitively, the capture sets do not matter for `mapFirst`, since parametricity forbids it from inspecting the actual values inside the pairs. If not for capture tunneling, we would need to annotate `p` as `Pair[A,B]^{cap}`, since both `A` and `B` and through them, `p` can capture arbitrary capabilities. In turn, this means that for the same reason, without tunneling we would also have `Pair[C,B]^{cap}` as the result type. This is of course unacceptably inaccurate.

Section 3 describes the foundational theory on which capture checking is based. It makes tunneling explicit through so-called *box* and *unbox* operations. Boxing hides a capture set, and unboxing recovers it. Boxed values need an explicit unbox operation before they can be accessed, and that unbox operation charges the capture set of the environment. If the unbox operation is part of a closure, the unboxed type's capture set will contribute to the captured variables of that closure. The need for such a mechanism is explained in more detail in Section 6.

The capture checker inserts virtual box and unbox operations based on actual and expected types similar to the way the type checker inserts implicit conversions. Boxing and unboxing has no runtime effect, so the insertion of these operations is only simulated, but not kept in the generated code.

2.6 Escape Checking

Following the principle of object capabilities, the universal capability `cap` should conceptually only be available as a parameter to the main program. Indeed, if it was available everywhere, capability checking would be undermined since one could mint new capabilities at will. In line with this reasoning, some capture sets are restricted and must not contain the universal capability.

Specifically, if a capturing type is an instance of a type variable, that capturing type is not allowed to carry the universal capability `{cap}`.³ There is a connection to tunneling here. The capture set of a type has to be present in the environment when a type is instantiated from a type variable. But `cap` is not itself available as a global entity in the environment. Hence, this should result in an error.

Using this principle, we can show why the introductory example in Section 1 reported an error. To recall, function `usingFile` was declared like this:

```
def usingFile[T](name: String, op: FileOutputStream^{cap} => T): T = ...
```

The capture checker rejects the illegal definition of `later`

```
val later = usingFile("out",
  f => (y: Int) => xs.foreach(x => f.write(x + y)))
```

with the following error message

```
|   val later = usingFile("out", f => (y: Int) => xs.foreach(x => f.write(x + y)))
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|The expression's type Int => Unit is not allowed to capture the root capability `cap`
|This usually means that a capability persists longer than its allowed lifetime.
```

This error message was produced by the following reasoning steps:

- Parameter `f` has type `FileOutputStream^{cap}`, which makes it a capability.

³This follows since type variables range over pure types, so `cap` must appear under a box. But rule (Box) in Figure 3 (shown later) restricts variables in boxed capture sets to be declared in the enclosing environment, which does not hold for `cap`.

- Therefore, the type of the expression

```
(y: Int) => xs.foreach(x => f.write(x + y))
```

is $\text{Int} \rightarrow \{f\} \text{Unit}$.

- Consequently, we assign the whole closure passed to `usingFile` the dependent function type $(f: \text{FileOutputStream})^{\{\text{cap}\}} \text{Int} \rightarrow \{f\} \text{Unit}$.
- The expected type of the closure is a simple, parametric, impure function type $\text{FileOutputStream}^{\{\text{cap}\}} \Rightarrow T$, for some instantiation of the type variable T .
- We cannot instantiate T with $\text{Int} \rightarrow \{f\} \text{Unit}$ since the expected function type is non-dependent. The smallest supertype that matches the expected type is thus $\text{FileOutputStream}^{\{\text{cap}\}} \Rightarrow \text{Int} \rightarrow \{\text{cap}\} \text{Unit}$.
- Hence, the type variable T is instantiated to $\text{Int} \rightarrow \{\text{cap}\} \text{Unit}$, which is not allowed and causes the error.

2.7 Escape Checking of Mutable Variables

Another way one could try to undermine capture checking would be to assign a closure with a local capability to a global variable. For instance, like this:⁴

```
var loophole: () -> {cap} Unit = () => ()
usingFile("tryEscape", f =>
  loophole = () => f.write(0)
)
loophole()
```

We prevent such scope extrusions by imposing the restriction that mutable variables cannot have types with universal capture sets.

One also needs to prevent returning or assigning a closure with a local capability in an argument of a parametric type. For instance, here is a slightly more refined attack:

```
val sneaky = usingFile { f => Pair(() => f.write(0), 1) }
sneaky.fst()
```

At the point where the `Pair` is created, the capture set of the first argument is $\{f\}$, which is OK. But at the point of use, it is $\{\text{cap}\}$: since f is no longer in scope, we need to widen the type to a supertype that does not mention it (c.f. the explanation of avoidance in Section 3.3). This causes an error, again, as the universal capability is not permitted to be in the unboxed form of the return type (c.f. the precondition of (UNBOX) in Figure 3, presented later).

3 THE $\text{CC}_{<,\square}$ CALCULUS

The syntax of $\text{CC}_{<,\square}$ is given in Figure 2. In short, it describes a dependently typed variant of System $\text{F}_{<}$ in **Monadic Normal Form (MNF)** with capturing types and boxes.

Dependently Typed. Types may refer to term variables in their capture sets, which introduces a simple form of (variable-)dependent typing. As a consequence, a function's result type may now refer to the parameter in its capture set. To be able to express this, the general form of a function type $\forall(x : U)T$ explicitly names the parameter x . We retain the non-dependent syntax $U \rightarrow T$ for function types as an abbreviation if the parameter is not mentioned in the result type T .

⁴Mutable variables are not covered by the formal treatment of $\text{CC}_{<,\square}$. We include the discussion anyway to show that escape checking can be generalized to scope extrusions separate from result values.

Variable	x, y, z, cap
Type Variable	X, Y, Z
Value	$v, w ::= \lambda(x : T) t \mid \lambda[X <: S] t \mid \boxed{x}$
Answer	$a ::= v \mid x$
Term	$t, s ::= a \mid x y \mid x [S] \mid \text{let } x = s \text{ in } t \mid C \multimap x$
Shape Type	$S, R ::= X \mid \top \mid \forall(x : U) T \mid \forall[X <: S] T \mid \boxed{T}$
Type	$T, U ::= S \mid S \wedge C$
Capture Set	$C ::= \{x_1, \dots, x_n\}$
Typing Context	$\Gamma, \Delta ::= \emptyset \mid \Gamma, X <: S \mid \Gamma, x : T \quad \text{if } x \neq \text{cap}$

Fig. 2. Syntax of system $\text{CC}_{<:\Box}$.

Dependent typing is attractive since it means that we can refer to object capabilities directly in types instead of having to go through auxiliary region or effect variables. We thus avoid clutter related to quantification of such auxiliary variables.

Monadic Normal Form. The term structure of $\text{CC}_{<:\Box}$ requires operands of applications to be variables. This does not constitute a loss of expressiveness, since a general application $t_1 t_2$ can be expressed as **let** $x_1 = t_1$ **in** **let** $x_2 = t_2$ **in** $x_1 x_2$. This syntactic convention has advantages for variable-dependent typing. In particular, typing function application in such a calculus requires substituting actual arguments for formal parameters. If arguments are restricted to be variables, these substitutions are just variable/variable renamings, which keep the general structure of a type. If arguments were arbitrary terms, such a substitution would in general map a type to something that was not syntactically a type. MNF [Hatcliff and Danvy 1994] is a slight generalization of the better-known A-normal form (ANF) [Sabry and Felleisen 1993] to allow arbitrary nesting of let expressions. We use here a variant of MNF where applications are over variables instead of values.

A similar restriction to MNF was employed in DOT [Amin et al. 2016], the foundation of Scala’s object model, for the same reasons. The restriction is invisible to source programs, which can still be in direct style. For instance, the Scala compiler selectively translates a source expression in direct style to MNF if a non-variable argument is passed to a dependent function. Type checking then takes place on the translated version.

Capturing Types. The types in $\text{CC}_{<:\Box}$ are stratified as *shape types* S and regular types T . Regular types can be shape types or capturing types $S \wedge \{x_1, \dots, x_n\}$. “ $\wedge$ ” has a higher precedence than $\Box$ or $\forall$ prefixes—for instance, $\forall(x : S) T \wedge C$ is read as $\forall(x : S) (T \wedge C)$. Shape types are made up from the usual type constructors in $F_{<:}$ plus boxes. We freely use shape types in place of types, assuming the equivalence $S \wedge \{\} \equiv S$.

Boxes. Type variables X can be bounded or instantiated only with shape types, not with regular types. To make up for this restriction, a regular type T can be encapsulated in a shape type by prefixing it with a box operator $\Box T$. On the term level, $\Box x$ injects a variable into a boxed type. A variable of boxed type is unboxed using the syntax $C \multimap x$ where C is the capture set of the underlying type of x . We saw in Section 2 that boxing and unboxing allow a kind of capability tunneling by omitting capabilities when values of parametric types are constructed and charging these capabilities instead at use sites.

System $F_{<:}$. We base $CC_{<:\square}$ on a standard type system that supports the two principal forms of polymorphism: subtyping and universal.

Subtyping comes naturally with capabilities in capture sets. First, a type capturing fewer capabilities is naturally a subtype of a type capturing more capabilities, and pure types are naturally subtypes of capturing types. Second, if capability x is derived from capability y , then a type capturing x can be seen as a subtype of the same type but capturing y .

Universal polymorphism poses specific challenges when capture sets are introduced which are addressed in $CC_{<:\square}$ by the stratification into shape types and regular types and the box/unbox operations that map between them.

Note that the only form of term dependencies in $CC_{<:\square}$ relate to capture sets in types. If we omit capture sets and boxes, the calculus is equivalent to standard $F_{<:}$, despite the different syntax. We highlight in the figures the essential additions with respect to $F_{<:}$ with a gray background.

$CC_{<:\square}$ is intentionally meant to be a small and canonical core calculus that does not cover higher-level features such as records, modules, objects, or classes. Although these features are certainly important, their specific details are also somewhat more varied and arbitrary than the core that is covered. Many different systems can be built on $CC_{<:\square}$, extending it with various constructs to organize code and data on higher levels.

Capture Sets. Capture sets C are finite sets of variables of the form $\{x_1, \dots, x_n\}$. We understand **cap** to be a special variable that can appear in capture sets but cannot be bound in Γ . We write $C \setminus x$ as a shorthand for subtraction of capture sets $C \setminus \{x\}$.

Capture sets of closures are determined using a function cv over terms.

Definition (Captured Variables). The captured variables $cv(t)$ of a term t are given as follows:

$$\begin{aligned}
 cv(\lambda(x : T)t) &= cv(t) \setminus x \\
 cv(\lambda[X <: S]t) &= cv(t) \\
 cv(x) &= \{x\} \\
 cv(\mathbf{let} \ x = v \ \mathbf{in} \ t) &= cv(t) && \text{if } x \notin cv(t) \\
 cv(\mathbf{let} \ x = s \ \mathbf{in} \ t) &= cv(s) \cup cv(t) \setminus x \\
 cv(x \ y) &= \{x, y\} \\
 cv(x[S]) &= \{x\} \\
 cv(\square x) &= \{\} \\
 cv(C \multimap x) &= C \cup \{x\}
 \end{aligned}$$

The definitions of captured and free variables of a term are very similar, with the following three differences:

- (1) Boxing a term $\square x$ obscures x as a captured variable.
- (2) Dually, unboxing a term $C \multimap x$ counts the variables in C as captured.
- (3) In an evaluated let binding $\mathbf{let} \ x = v \ \mathbf{in} \ t$, the captured variables of v are counted only if x is a captured variable of t .

The first two rules encapsulate the essence of box-unbox pairs: boxing a term obscures its captured variable and makes it necessary to unbox the term before its value can be accessed; unboxing a term presents variables that were obscured when boxing. The third rule is motivated by the case where a variable x is bound to a value v ; then we do not want to count the captured variables of v if x is either boxed or not mentioned at all in the let body. The intuition behind this rule is that such variables would naturally be disregarded if $CC_{<:\square}$ was not in MNF.

Figure 3 presents typing and evaluation rules for $CC_{<:\square}$. There are four main sections on subcapturing, subtyping, typing, and evaluation. These are explained in the following.

Subcapturing

$$\begin{array}{c}
\boxed{\Gamma \vdash C <: C} \\
\frac{x \in C}{\Gamma \vdash \{x\} <: C} \text{ (SC-ELEM)} \quad \frac{\Gamma \vdash \{x_1\} <: C \dots \Gamma \vdash \{x_n\} <: C}{\Gamma \vdash \{x_1, \dots, x_n\} <: C} \text{ (SC-SET)} \\
\frac{x : S^\wedge C_1 \in \Gamma \quad \Gamma \vdash C_1 <: C}{\Gamma \vdash \{x\} <: C} \text{ (SC-VAR)}
\end{array}$$

Subtyping

$$\begin{array}{c}
\boxed{\Gamma \vdash T <: T} \\
\frac{\Gamma \vdash T <: T}{\Gamma \vdash T <: T} \text{ (REFL)} \quad \frac{\Gamma \vdash T_1 <: T_2 \quad \Gamma \vdash T_2 <: T_3 \quad \Gamma \vdash T_2 \mathbf{wf}}{\Gamma \vdash T_1 <: T_3} \text{ (TRANS)} \\
\frac{X <: S \in \Gamma}{\Gamma \vdash X <: S} \text{ (TVAR)} \quad \frac{}{\Gamma \vdash S <: \top} \text{ (TOP)} \\
\frac{\Gamma \vdash U_2 <: U_1 \quad \Gamma, x : U_2 \vdash T_1 <: T_2}{\Gamma \vdash \forall(x : U_1)T_1 <: \forall(x : U_2)T_2} \text{ (FUN)} \quad \frac{\Gamma \vdash S_2 <: S_1 \quad \Gamma, X <: S_2 \vdash T_1 <: T_2}{\Gamma \vdash \forall[X <: S_1]T_1 <: \forall[X <: S_2]T_2} \text{ (TFUN)} \\
\frac{\Gamma \vdash C_1 <: C_2 \quad \Gamma \vdash S_1 <: S_2}{\Gamma \vdash S_1^\wedge C_1 <: S_2^\wedge C_2} \text{ (CAPT)} \quad \frac{\Gamma \vdash T_1 <: T_2}{\Gamma \vdash \Box T_1 <: \Box T_2} \text{ (BOXED)}
\end{array}$$

Typing

$$\begin{array}{c}
\boxed{\Gamma \vdash t : T} \\
\frac{x : S^\wedge C \in \Gamma}{\Gamma \vdash x : S^\wedge \{x\}} \text{ (VAR)} \quad \frac{\Gamma \vdash t : T \quad \Gamma \vdash T <: U \quad \Gamma \vdash U \mathbf{wf}}{\Gamma \vdash t : U} \text{ (SUB)} \\
\frac{\Gamma, x : U \vdash t : T \quad \Gamma \vdash U \mathbf{wf}}{\Gamma \vdash \lambda(x : U)t : (\forall(x : U)T)^\wedge \mathbf{cv}(t) \setminus x} \text{ (ABS)} \quad \frac{\Gamma, X <: S \vdash t : T \quad \Gamma \vdash S \mathbf{wf}}{\Gamma \vdash \lambda[X <: S]t : (\forall[X <: S]T)^\wedge \mathbf{cv}(t)} \text{ (TABS)} \\
\frac{\Gamma \vdash x : (\forall(z : U)T)^\wedge C \quad \Gamma \vdash y : U}{\Gamma \vdash x y : [z := y]T} \text{ (APP)} \quad \frac{\Gamma \vdash x : (\forall[X <: S]T)^\wedge C}{\Gamma \vdash x [S] : [X := S]T} \text{ (TAPP)} \\
\frac{\Gamma \vdash x : S^\wedge C \quad C \subseteq \text{dom}(\Gamma)}{\Gamma \vdash \Box x : \Box S^\wedge C} \text{ (BOX)} \quad \frac{\Gamma \vdash x : \Box S^\wedge C \quad C \subseteq \text{dom}(\Gamma)}{\Gamma \vdash C \multimap x : S^\wedge C} \text{ (UNBOX)} \\
\frac{\Gamma \vdash s : T \quad \Gamma, x : T \vdash t : U \quad x \notin \text{fv}(U)}{\Gamma \vdash \mathbf{let } x = s \mathbf{ in } t : U} \text{ (LET)}
\end{array}$$

Evaluation

$$\begin{array}{c}
\boxed{\Gamma \vdash t \longrightarrow t'} \\
\begin{array}{lll}
\sigma[e[x y]] & \longrightarrow & \sigma[e[[z := y]t]] \quad \text{if } \sigma(x) = \lambda(z : T)t \quad \text{(APPLY)} \\
\sigma[e[x [S]]] & \longrightarrow & \sigma[e[[X := S]t]] \quad \text{if } \sigma(x) = \lambda[X <: S']t \quad \text{(TAPPLY)} \\
\sigma[e[\Box \multimap x]] & \longrightarrow & \sigma[e[\Box y]] \quad \text{if } \sigma(x) = \Box y \quad \text{(OPEN)} \\
\sigma[e[\mathbf{let } x = y \mathbf{ in } t]] & \longrightarrow & \sigma[e[[x := y]t]] \quad \text{(RENAME)} \\
\sigma[e[\mathbf{let } x = v \mathbf{ in } t]] & \longrightarrow & \sigma[\mathbf{let } x = v \mathbf{ in } e[t]] \quad \text{if } e \neq [] \quad \text{(LIFT)}
\end{array} \\
\text{where Store context } \sigma ::= [] \mid \mathbf{let } x = v \mathbf{ in } \sigma \\
\text{Eval context } e ::= [] \mid \mathbf{let } x = e \mathbf{ in } t
\end{array}$$

Fig. 3. Typing and evaluation rules of system $\text{CC}_{<:\Box}$.

3.1 Subcapturing

Subcapturing establishes a preorder relation on capture sets that gets propagated to types. Smaller capture sets with respect to subcapturing lead to smaller types with respect to subtyping.

The relation is defined by three rules. The first two rules (SC-SET) and (SC-ELEM) establish that subsets imply subcaptures. In other words, smaller capture sets subcapture larger ones. The last rule (SC-VAR) is the most interesting since it reflects an essential property of object capabilities. It states that a variable x of capturing type $S \wedge C$ generates a capture set $\{x\}$ that subcaptures the capabilities C with which the variable was declared. In a sense, (SC-VAR) states a monotonicity property: a capability refines the capabilities from which it is created. In particular, capabilities cannot be created from nothing. Every capability needs to be derived from some more sweeping capabilities which it captures in its type.

The rule also validates our definition of capabilities as variables with non-empty capture sets in their types. Indeed, if a variable is defined as $x : S \wedge \{\}$, then by (SC-VAR) we have $\{x\} <: \{\}$. This means that the variable can be disregarded in the formation of cv, for instance. Even if x occurs in a term, a capture set with x in it is equivalent (with respect to mutual subcapturing) to a capture set without. Hence, x can safely be dropped without affecting subtyping or typing.

Rules (SC-SET) and (SC-ELEM) mean that if set C is a subset of C' , we also have $C <: C'$. But the reverse is not true. For instance, with (SC-VAR) we can derive the following relationship assuming lambda-bound variables x and y :

$$x : T^\wedge \{\mathbf{cap}\}, y : T^\wedge \{x\} \vdash \{y\} <: \{x\}.$$

Intuitively this makes sense, as y can capture no more than x . However, we *cannot* derive $\{x\} <: \{y\}$, since arguments passed for y may in fact capture *less* than x —for example, they could be pure.

Although there are no subcapturing rules for top or bottom capture sets, we can still establish the following proposition.

PROPOSITION 3.1. *If C is well-formed in Γ , then $\Gamma \vdash \{\} <: C <: \{\mathbf{cap}\}$.*

A proof is enclosed in the appendix.

PROPOSITION 3.2. *The subcapturing relation $\Gamma \vdash _ <: _$ is a preorder.*

PROOF. We can show that transitivity and reflexivity are admissible. □

3.2 Subtyping

The subtyping rules of $\text{CC}_{<:\square}$ are very similar to those of System $F_{<:}$, with the only significant addition being the rules for capturing and boxed types. Note that as $S \equiv S \wedge \{\}$, both transitivity and reflexivity apply to shape types as well. Rule (CAPT) allows comparing types that have capture sets, where smaller capture sets lead to smaller types. Rule (BOXED) propagates subtyping relations between types to their boxed versions.

3.3 Typing

Typing rules are again close to System $F_{<:}$, with differences to account for capture sets.

Rule (VAR) is the basis for capability refinements. If x is declared with type $S \wedge C$, then the type of x has $\{x\}$ as its capture set instead of C . The capture set $\{x\}$ is more specific than C , in the subcapturing sense. Therefore, we can recover the capture set C through subsumption.

Rules (ABS) and (TABS) augment the abstraction's type with a capture set that contains the captured variables of the term. Through subsumption and rule (SC-VAR), untracked variables can immediately be removed from this set.

Type well-formedness		$\boxed{\Gamma \vdash C \text{ wf}}$	$\boxed{\Gamma \vdash T \text{ wf}}$
$\frac{C \subseteq \text{dom}(\Gamma) \cup \{\text{cap}\}}{\Gamma \vdash C \text{ wf}}$	(WF-CSET)	$\frac{\Gamma \vdash C \text{ wf} \quad \Gamma \vdash S \text{ wf}}{\Gamma \vdash S^\wedge C \text{ wf}}$	(WF-CAPT)
$\frac{X <: S \in \Gamma}{\Gamma \vdash X \text{ wf}}$	(WF-TVAR)	$\frac{\Gamma \vdash T \text{ wf}}{\Gamma \vdash \Box T}$	(WF-BOXED)
$\frac{\Gamma \vdash U \text{ wf} \quad \Gamma, x : U \vdash T \text{ wf}}{\Gamma \vdash \forall(x : U)T \text{ wf}}$	(WF-FUN)	$\frac{\Gamma \vdash S \text{ wf} \quad \Gamma, X <: S \vdash T \text{ wf}}{\Gamma \vdash \forall[X <: S]T \text{ wf}}$	(WF-TFUN)
$\Gamma \vdash \top \text{ wf} \quad (\text{WF-TOP})$			

Fig. 4. Type well-formedness rules of system $\text{CC}_{<:\Box}$.

The (APP) rule substitutes references to the function parameter with the argument to the function. This is possible since arguments are guaranteed to be variables. The function's capture set C is disregarded, reflecting the principle that the function closure is consumed by the application. Rule (TAPP) is analogous.

Aside. A more conventional version of (TAPP) would be

$$\frac{\Gamma \vdash x : (\forall[X <: S']T)^\wedge C \quad \Gamma \vdash S <: S'}{\Gamma \vdash x[S] : [X := S]T} \quad (\text{TAPP}')$$

That formulation is equivalent to (TAPP) in the sense that either rule is derivable from the other, using subsumption and contravariance of type bounds.

Rules (BOX) and (UNBOX) map between boxed and unboxed types. They require all members of the capture set under the box to be bound in the environment Γ . Consequently, although one can create a boxed type with $\{\text{cap}\}$ as its capture set through subsumption, one cannot unbox values of this type. This property is fundamental for ensuring scoping of capabilities.

Avoidance. As is usual in dependent type systems, Rule (LET) has as a side condition that the bound variable x does not appear free in the result type U . This so called *avoidance* property is usually attained through subsumption. For instance, consider an enclosing capability $c : T_1$ and the following term:

$$\text{let } x = \lambda(y : T_2)c \text{ in } \lambda(z : T_3^\wedge \{x\})z.$$

The most specific type of x is $(\forall(y : T_2)T_1)^\wedge \{c\}$, and the most specific type of the body of the let is $\forall(z : T_3^\wedge \{x\})T_3^\wedge \{z\}$. We need to find a supertype of the latter type that does not mention x . It turns out the most specific such type is $(\forall(y : T_3)T_3)^\wedge \{c\}$, so that is a possible type of the let, and it should be the inferred type.

In general, there is always a most specific avoiding type for a (LET), as we prove in Appendix A.7.

PROPOSITION 3.3. *Consider a term $\text{let } x = s \text{ in } t$ in an environment Γ such that $\Gamma \vdash s : T_1$ and $\Gamma, x : T_1 \vdash t : T_2$. Then there exists a minimal (wrt $<:$) type T_3 such that $T_2 <: T_3$ and $x \notin \text{fv}(T_3)$.*

3.4 Well-Formedness

Well-formedness $\Gamma \vdash T \text{ wf}$ is equivalent to well-formedness in System $\text{F}_{<:}$, in that free variables in types and terms must be defined in the environment, except that capturing types may mention the universal capability cap in their capture sets. We present the well-formedness rules in Figure 4.

$$\frac{\Gamma \vdash v : T \quad \Gamma, x : T \vdash \sigma \sim \Delta}{\Gamma \vdash \mathbf{let} x = v \mathbf{in} \sigma \sim x : T, \Delta} \quad \Gamma \vdash [] \sim \cdot$$

Fig. 5. Matching environment $\boxed{\Gamma \vdash \sigma \sim \Delta}$.

3.5 Evaluation

Evaluation is defined by a small-step reduction relation. This relation is quite different from usual reduction via term substitution. Substituting values for variables would break the MNF of a program. Instead, we reduce the right-hand sides of let-bound variables in place and lookup the bindings in the environment of a redex.

Every redex is embedded in an outer *store context* and an inner *evaluation context*. These represent orthogonal decompositions of let bindings. An evaluation context e always puts the focus $[]$ on the right-hand side t_1 of a let binding $\mathbf{let} x = t_1 \mathbf{in} t_2$. By contrast, a store context σ puts the focus on the following term t_2 and requires that t_1 is evaluated.

The first three rules—(APPLY), (TAPPLY), (OPEN)—rewrite simple redexes: applications, type applications, and unboxings. Each of these rules looks up a variable in the enclosing store and proceeds based on the value that was found.

The last two rules are administrative in nature. They both deal with evaluated **lets** in redex position. If the right-hand side of the **let** is a variable, the **let** gets expanded out by renaming the bound variable using (RENAME). If it is a value, the **let** gets lifted out into the store context using (LIFT).

PROPOSITION 3.4. *Evaluation is deterministic. If $t \longrightarrow u_1$ and $t \longrightarrow u_2$, then $u_1 = u_2$.*

PROOF. By a straightforward inspection of the reduction rules and definitions of contexts. $\square$

4 METATHEORY

We prove that $\text{CC}_{<,\square}$ is sound through the standard progress and preservation theorems. The proofs for all the lemmas and theorems stated in this section are provided in the appendix. Progress and preservation and the capture prediction lemma for the calculus have also been mechanized by Fourment and Xu [2023].

We follow the Barendregt convention and only consider typing contexts where all variables are unique: for all contexts of the form $\Gamma, x : T$, we have $x \notin \text{dom}(\Gamma)$.

To prove both progress and preservation, we need technical lemmas that allow manipulation of typing judgments for terms under store and evaluation contexts. To state these lemmas, we first need to define what it means for typing and store contexts to match, which we do in Figure 5.

Having $\Gamma \vdash \sigma \sim \Delta$ lets us know that σ is well-typed in Γ if we use Δ as the types of the bindings. Using this definition, we can state the following four lemmas, which also illustrate how the store and evaluation contexts interact with typing.

Definition 4.1 (Evaluation Context Typing ($\Gamma \vdash e : U \Rightarrow T$)). We say that e can be typed as $U \Rightarrow T$ in Γ iff for all t such that $\Gamma \vdash t : U$, we have $\Gamma \vdash e[t] : T$.

LEMMA 4.2 (EVALUATION CONTEXT TYPING INVERSION). $\Gamma \vdash e[s] : T$ implies that for some U , we have $\Gamma \vdash e : U \Rightarrow T$ and $\Gamma \vdash s : U$.

LEMMA 4.3 (EVALUATION CONTEXT REIFICATION). If both $\Gamma \vdash e : U \Rightarrow T$ and $\Gamma \vdash s : U$, then $\Gamma \vdash e[s] : T$.

LEMMA 4.4 (STORE CONTEXT TYPING INVERSION). $\Gamma \vdash \sigma[t] : T$ implies that for some Δ , we have $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$.

LEMMA 4.5 (STORE CONTEXT REIFICATION). *If both $\Gamma, \Delta \vdash t : T$ and $\Gamma \vdash \sigma \sim \Delta$, then also $\Gamma \vdash \sigma[t] : T$.*

We can now proceed to our main theorems; their statements differ slightly from System $F_{<}$, as we need to account for MNF. Our preservation theorem captures that the important type to preserve is the one assigned to the term under the store. It is stated as follows.

THEOREM 4.6 (PRESERVATION). *If we have $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, then $\sigma[t] \longrightarrow \sigma[t']$ implies that $\Gamma, \Delta \vdash t' : T$.*

Before stating the progress theorem, we need an auxiliary definition.

Definition 4.7 (Proper Configuration). A term form $\sigma[t]$ is a *proper configuration* if t is not of the form **let** $x = v$ **in** t' .

THEOREM 4.8 (PROGRESS). *If $\vdash \sigma[t] : T$ and $\sigma[t]$ is a proper configuration, then either t is an answer a or $\sigma[t] \longrightarrow \sigma[t']$ for some t' .*

The lemmas needed to prove progress and preservation are for the most part standard. As our calculus is term-dependent, we also need to account for term substitution affecting both environments and types, not only terms. For instance, the lemma stating that term substitution preserves typing is expressed as follows.

LEMMA 4.9 (TERM SUBSTITUTION PRESERVES TYPING). *If $\Gamma, x : U, \Delta \vdash t : T$ and $\Gamma \vdash y : U$, then $\Gamma, [x := y]\Delta \vdash [x := y]t : [x := y]T$.*

In this statement, we can also see that we only consider substituting one term variable for another, due to MNF. Using MNF affects other parts of the proof as well—in addition to typical canonical forms lemmas, we also need to show that looking up the value bound to a variable in a store preserves the types we can assign to the variable.

LEMMA 4.10 (VARIABLE LOOKUP INVERSION). *If we have both $\Gamma \vdash \sigma \sim \Delta$ and $x : S^\wedge C \in \Gamma, \Delta$, then $\sigma(x) = v$ implies that $\Gamma, \Delta \vdash v : S^\wedge C$.*

Capture Sets and Captured Variables. Our typing rules use cv to calculate the capture set that should be assigned to terms. With that in mind, we can ask this question: what is the exact relationship between captured variables and capture sets we use to type the terms?

Because of subcapturing, this relationship is not as obvious as it might seem. For fully evaluated terms (of the form $\sigma[a]$), their captured variables are the most precise capture set they can be assigned. The following lemma states this formally.

LEMMA 4.11 (CAPTURE PREDICTION FOR ANSWERS). *If $\Gamma \vdash \sigma[a] : S^\wedge C$, then $\Gamma \vdash \text{cv}(\sigma[a]) <: C$.*

If we start with an unreduced term $\sigma[t]$, then the situation becomes more complex. It can mention and use capabilities that will not be reflected in the capture set at all—for instance, if $t = xy$, the capture set of x is irrelevant to the type assigned to t by (APP). However, if $\sigma[t]$ reduces fully to a term of the form $\sigma'[\sigma[a]]$, the captured variables of $\sigma'[a]$ will correspond to capture sets we could assign to t .

In other words, the capture sets we assign to unevaluated terms under a store context predict variables that will be captured by the answer those terms reduce to. Formally, we can express this as follows.

LEMMA 4.12 (CAPTURE PREDICTION FOR TERMS). *Let $\vdash \sigma \sim \Delta$ and $\Delta \vdash t : S^\wedge C$. Then $\sigma[t] \longrightarrow^* \sigma'[\sigma[a]]$ implies that $\Delta \vdash \text{cv}(\sigma'[a]) <: C$.*

4.1 Predicting Used Capabilities

In this section, we develop an additional correctness criterion: a theorem that uses capture sets to predict what capabilities may be used while a term is evaluated. Since the ability to perform effects is mediated by capabilities in capability-safe systems, predicting what capabilities may be used by terms is the same as reasoning about the *authority* of terms to perform side-effectful operations [Drossopoulou et al. 2016; Miller 2006]. This theorem is also an important correctness criterion for boxing, as we will discuss later.

If we want to reason about what capabilities are used, we need to have a concept of primitive capabilities which must be tracked, not unlike how STLC needs base types [Pierce 2002] to make its correctness theorem non-vacuous. Although object capabilities come in many forms, for our current purposes it suffices to consider capabilities that exist for the entire duration of the program, such as a capability to access the filesystem or the standard output. Within our base system, we can simply designate an outer fragment of the store as the *platform* context Ψ , which introduces well-behaved primitive capabilities:

$$\Psi ::= [] \mid \mathbf{let} \, x = v \mathbf{in} \, \Psi \quad \mathbf{if} \, \text{fv}(v) = \{\}. \quad \text{if } \text{fv}(v) = \{\}.$$

The operational semantics of the capabilities in Ψ are defined by the values v . The values need to be closed, as otherwise the capabilities would depend on other capabilities and would not be primitive. Since Ψ binds capabilities, their capture set should be $\{\mathbf{cap}\}$.

Definition (Well-Typed Program). A term $\Psi[t]$ is a well-typed program if we have $\Delta \vdash t : T$ for some Δ such that $\vdash \Psi \sim \Delta$ and for all $x \in \Delta$ there exists a shape type S such that $x : S^\wedge \{\mathbf{cap}\} \in \Delta$.

We can now state an intermediate lemma necessary to prove our correctness criterion.

LEMMA 4.13 (PROGRAM AUTHORITY PRESERVATION). *Let $\Psi[t] \longrightarrow \Psi[t']$, where $\Psi[t]$ is a well-typed program. Then $\text{cv}(t')$ is a subset of $\text{cv}(t)$.*

We now formally state what capabilities are used during evaluation. Since Ψ only binds abstractions, it makes sense to say a capability x is used if during evaluation we reduced an application form.

Definition (Used Capabilities).

$$\begin{aligned} \text{used}(t_1 \longrightarrow t_2 \longrightarrow \cdots \longrightarrow t_n) &= \text{used}(t_1 \longrightarrow t_2) \cup \text{used}(t_2 \longrightarrow \cdots \longrightarrow t_n) \\ \text{used}(\sigma[e[xy]] \longrightarrow \sigma[t]) &= \{x\} \\ \text{used}(\sigma[e[x[S]]] \longrightarrow \sigma[t]) &= \{x\} \\ \text{used}(t_1 \longrightarrow t_2) &= \{\} \quad (\text{otherwise}) \end{aligned}$$

The last case applies to rules (OPEN), (RENAME), (LIFT).

We are ready to state the theorem.

THEOREM 4.14 (USED CAPABILITY PREDICTION). *Let $\Psi[t] \longrightarrow^* \Psi[t']$, where $\Psi[t]$ is a well-typed program. Then the primitive capabilities used during the reduction are a subset of the authority of t :*

$$\{x \mid x \in \text{used}(\Psi[t] \longrightarrow^* \Psi[t']), x \in \text{dom}(\Psi)\} \subseteq \text{cv}(t).$$

4.2 Correctness of Boxing

Both Lemma 4.13 and Theorem 4.14 would be trivially true if $\text{cv}(t)$ was just the free variables of t , since evaluation typically does not add new free variables to a term. However, boxes allow

$$\begin{aligned}
& \text{fs} : \text{Fs}^\wedge \{\mathbf{cap}\} \vdash \lambda(f : \text{Proc}^\wedge \{\text{fs}\}) f () && : \forall(f : \text{Proc}^\wedge \{\text{fs}\}) \text{Unit} \\
& \text{fs} : \text{Fs}^\wedge \{\mathbf{cap}\} \vdash \lambda(f : \Box \text{Proc}^\wedge \{\text{fs}\}) \mathbf{let} g = \{\text{fs}\} \multimap f \mathbf{in} g () && : (\forall(f : \Box \text{Proc}^\wedge \{\text{fs}\}) \text{Unit})^\wedge \{\text{fs}\}
\end{aligned}$$

Fig. 6. An example of boxes shifting what capture sets are charged with capabilities.

preventing some captured free variables from appearing in capture sets. For instance, if we first box x and then pass it as an argument to f , the overall cv will not mention x :

$$\text{cv}(\mathbf{let} y = \Box x \mathbf{in} f y) = \{f\}.$$

Given this behavior, what is the correctness criterion for how we type box and unbox forms? Intuitively, we should be unable to “smuggle in” a capability via boxes: a term’s capabilities should all be accounted for. By the progress theorem and a straightforward induction, we can prove that the cv of a term that boxes and immediately unboxes a capability accounts for the unboxed capability.

PROPOSITION 4.15. *Let $\vdash \sigma \sim \Delta$ and $t = (\mathbf{let} y = \Box x \mathbf{in} C \multimap y)$ such that we have $\Delta \vdash e[t] : T$ for some e and T . Then $\text{cv}(t) = C$ and we also have*

$$\Delta \vdash \{x\} <: C.$$

Speaking more generally, the fundamental function of boxes is that they allow *temporarily* preventing a captured free variable from affecting the cv of a term. The capability inside the box can still be used via the unbox form $C \multimap x$, but only at the cost of adding C , the “key” used to open the box, to the cv of the term. The correctness criterion for box and unbox forms is that the keys used to open boxes should account for the capabilities inside the box: a term should only be able to use capabilities that are accounted for by its cv, just as Lemma 4.13 and Theorem 4.14 show.⁵

There is another aspect of boxing explained by these theorems: boxes can later be opened with unbox forms, shifting where capture sets appear. As an example, consider the following two lambdas, both of which may use fs (we define $\text{Proc} \triangleq \forall(x : \text{Unit}) \text{Unit}$): the first lambda’s argument is a capability—a closure capturing fs . The lambda can invoke this closure without affecting its capture set. Meanwhile, the argument of the second lambda is *pure*: a box containing a closure capturing fs . The second lambda can still invoke its argument, but only after unboxing it, which charges its capture set with the fs capability.

Understanding that capture sets describe the authority of terms explains why it is sound for boxes to shift a capability from one capture set to another. To illustrate, let Γ bind the first closure from Figure 6 as $f_1 : \forall(g : \text{Proc}^\wedge \{\text{fs}\}) \text{Unit}$ and the second closure as $f_2 : (\forall(g : \Box \text{Proc}^\wedge \{\text{fs}\}) \text{Unit})^\wedge \{\text{fs}\}$ and also bind an fs -capturing procedure as $p : \text{Proc}^\wedge \{\text{fs}\}$. Calling either f_1 or f_2 can use fs , which is reflected by cv even if the capture sets of f_1 and f_2 are different. In the first case, we have $\Gamma \vdash \text{cv}(f_1 p) <: \{p\} <: \{\text{fs}\}$: we can elide f_1 from the capture set, but afterward the smallest set we can widen to is $\{\text{fs}\}$. In the second case, we have $\Gamma \vdash \text{cv}(\mathbf{let} p' = \Box p \mathbf{in} f_2 p') = \{f_2\} <: \{\text{fs}\}$: p is absent from the cv, but the smallest capture set to which we can widen $\{f_2\}$ is still $\{\text{fs}\}$. We correctly predict the authority of both terms.

When we refer to untracked closures, such as $f : (\forall(x : \text{Unit}) \text{Unit})^\wedge \{\}$, as *pure*, we are also indirectly using the notion that a term’s cv reflects its authority. What we mean is that such closures cannot be used to cause any effects on their own. Formally, when we reduce $f ()$ to $[x := ()]t$, based on (ABS) we must have $\text{cv}([x := ()]t) = \{\}$: a term that cannot use any capabilities.

⁵Specifically, the (OPEN) case in the proof of Lemma 4.13 relies on the boxed capability subcapturing the unboxing key, allowing Lemma A.56 to be used.

5 EXAMPLES

We have implemented a type checker for $CC_{<,\Box}$ as an extension of the Scala 3 compiler to enable experimentation with larger code examples. Notably, our extension infers which types must be boxed, and automatically generates boxing and unboxing operations when values are passed to and returned from instantiated generic datatypes, so none of these technical details appear in the actual user-written Scala code. We now present examples that demonstrate the usability of the language.

5.1 Church-Encoded Lists

In this section, we remain close to the core calculus by encoding lists using only functions; here, we still show the boxed types and boxing and unboxing operations that the compiler infers in gray, although they are not in the source code.

Using the Scala prototype implementation of $CC_{<,\Box}$, the encoding by Böhm and Berarducci [1985] of a linked list data structure can be implemented and typed as follows. Here, a list is represented by its right fold function:

```
type Op[T <:  $\Box$  Any{cap}], C <:  $\Box$  Any{cap}] =
  (v: T) => (s: C) => C

type List[T <:  $\Box$  Any{cap}] =
  [C <:  $\Box$  Any{cap}] -> (op: Op[T, C]) ->{op} (s: C) -> C

def nil[T <:  $\Box$  Any{cap}]: List[T] =
  [C <:  $\Box$  Any{cap}] => (op: Op[T, C]) => (s: C) => s

def cons[T <:  $\Box$  Any{cap}](hd: T, tl: List[T]): List[T] =
  [C <:  $\Box$  Any{cap}] => (op: Op[T, C]) => (s: C) => op(hd)(tl[C](op)(s))
```

A list inherently captures any capabilities that may be captured by its elements. Therefore, naively, one may expect the capture set of the list to include the capture set of the type T of its elements. However, boxing and unboxing enables us to elide the capture set of the elements from the capture set of the containing list. When constructing a list using `cons`, the elements must be boxed:

```
cons( $\Box$  1, cons( $\Box$  2, cons( $\Box$  3, nil)))
```

A `map` function over the list can be implemented and typed as follows:

```
def map[A <:  $\Box$  Any{cap}, B <:  $\Box$  Any{cap}](xs: List[A])(f: A ->{cap} B)
  : List[B]
  = xs[List[B]]((hd: A) => (tl: List[B]) => cons(f(hd), tl))(nil)
```

The mapped function f may capture any capabilities, as documented by the capture set $\{cap\}$ in its type. However, this does not affect the type of `map` or its result type `List[B]`, since the mapping is strict, so the resulting list does not capture any capabilities captured by f . If a value returned by the function f were to capture capabilities, this would be reflected in its type, the concrete type substituted for the type variable B , and would therefore be reflected in the concrete instantiation of the result type `List[B]` of `map`.

5.2 Stack Allocation

In this and the following section, we use additional Scala features in larger examples to implement stack allocation and polymorphic data structures. For these examples, we present the source

code without cluttering it with the boxing operations inferred by the compiler. Furthermore, we use the abbreviation that a single trailing `^` that is not followed by an opening brace stands for `{cap}`.

Automatic memory management using a garbage collector is convenient and prevents many errors, but it can impose significant performance overheads in programs that need to allocate large numbers of short-lived objects. If we can bound the lifetimes of some objects to coincide with a static scope, it is much cheaper to allocate those objects on a stack as follows:⁶

```
class Pooled

val stack = mutable.ArrayBuffer[Pooled]()
var nextFree = 0

def withFreshPooled[T](op: Pooled => T): T =
  if nextFree >= stack.size then stack.append(new Pooled)
  val pooled = stack(nextFree)
  nextFree = nextFree + 1
  val ret = op(pooled)
  nextFree = nextFree - 1
  ret
```

The `withFreshPooled` method calls the provided function `op` with a freshly stack-allocated instance of class `Pooled`. It works as follows. The `stack` maintains a pool of already allocated instances of `Pooled`. The `nextFree` variable records the offset of the first element of `stack` that is available to reuse; elements before it are in use. The `withFreshPooled` method first checks whether the `stack` has any available instances; if not, it adds one to the `stack`. Then it increments `nextFree` to mark the first available instance as used, calls `op` with the instance, and decrements `nextFree` to mark the instance as freed. In the fast path, allocating and freeing an instance of `Pooled` is reduced to just incrementing and decrementing the integer `nextFree`.

However, this mechanism fails if the instance of `Pooled` outlives the execution of `op`, if `op` captures it in its result. Then the captured instance may still be accessed while at the same time also being reused by later executions of `op`. For example, the following invocation of `withFreshPooled` returns a closure that accesses the `Pooled` instance when it is invoked on the second line, after the `Pooled` instance has been freed:

```
val pooledClosure = withFreshPooled(pooled => () => pooled.toString)
pooledClosure()
```

Using capture sets, we can prevent such captures and ensure the safety of stack allocation just by marking the `Pooled` instance as tracked:

```
def withFreshPooled[T](op: Pooled^ => T): T =
```

Now the `pooled` instance can be captured only in values whose capture set accounts for `{pooled}`. The type variable `T` cannot be instantiated with such a capture set because `pooled` is not in scope outside of `withFreshPooled`, so only `{cap}` would account for `{pooled}`, but we disallowed instantiating a type variable with `{cap}`. With this declaration of `withFreshPooled`, the preceding `pooledClosure` example is correctly rejected, whereas the following safe example is allowed:

```
withFreshPooled(pooled => pooled.toString)
```

⁶For simplicity, this example is neither thread nor exception safe.

5.3 Collections

In the following examples, we show that a typing discipline based on $CC_{<:\Box}$ can be lightweight enough to make capture checking of operations on standard collection types practical. This is important, since such operations are the backbone of many programs. All examples compile with our current capture checking prototype [Scala 2022b].

We contrast the APIs of common operations on Scala's standard collection types `List` and `Iterator` when capture sets are taken into account. Both APIs are expressed as Scala 3 extension methods [Odersky and Martres 2020] over their first parameter. Here is the `List` API:

```
extension [A](xs: List[A])
  def apply(n: Int): A
  def foldLeft[B](z: B)(op: (B, A) => B): B
  def foldRight[B](z: B)(op: (A, B) => B): B
  def foreach(f: A => Unit): Unit
  def iterator: Iterator[A]
  def drop(n: Int): List[A]
  def map[B](f: A => B): List[B]
  def flatMap[B](f: A => IterableOnce[B]^): List[B]
  def ++[B >: A](xs: IterableOnce[B]^): List[B]
```

Notably, these methods have almost exactly the same signatures as their versions in the standard Scala collections library. The only differences concern the arguments to `flatMap` and `++` which now admit an `IterableOnce` argument with an arbitrary capture set. The type `IterableOnce[B]^` makes a subtle distinction: this collection may perform computation to produce elements of type `B`, and it may have captured capabilities to perform this computation as denoted by the “`^`”. All these capabilities will have been used (and therefore discarded) by the time the resulting `List[B]` is produced. Of course, we could have left out the trailing “`^`”s, but this would have needlessly restricted the argument to non-capturing collections.

Contrast this with some of the same methods for iterators:

```
extension [A](it: Iterator[A]^{it})
  def apply(n: Int): A
  def foldLeft[B](z: B)(op: (B, A) => B): B
  def foldRight[B](z: B)(op: (A, B) => B): B
  def foreach(f: A => Unit): Unit
  def drop(n: Int): Iterator[A]^{it}
  def map[B](f: A => B): Iterator[B]^{it, f}
  def flatMap[B](f: A => IterableOnce[B]^): Iterator[B]^{it, f}
  def ++[B >: A](xs: IterableOnce[B]^): Iterator[B]^{it, xs}
```

Here, methods `apply`, `foldLeft`, `foldRight`, and `foreach` again have the same signatures as in the current Scala standard library. But the remaining four operations need additional capture annotations. Method `drop` on iterators returns the given iterator `it` after skipping `n` elements. Consequently, its result has `{it}` as capture set. Methods `map` and `flatMap` lazily map elements of the current iterator as the result is traversed. Consequently, they retain both `it` and `f` in their result capture set. Method `++` concatenates two iterators and therefore retains both of them in its result capture set.

The examples attest to the practicality of capture checking. Method signatures are generally concise. Higher-order methods over strict collections by and large keep the same types as before. Capture annotations are only needed for capabilities that are retained in closures and are executed

on demand later, which matches the developer's intuitive understanding of reference patterns and signal information that is relevant in this context.

6 WHY BOXES?

Boxed types and box/unbox operations are a key part of the calculus to make type abstraction work. This might seem surprising. After all, as long as the capture set is not the root capture set $\{\text{cap}\}$, one can always go from a capturing type to its boxed type and back by boxing and unboxing operations. So in what sense is this more than administrative ceremony? The key observation here is that an unbox operation $C \multimap x$ charges the capture set of the environment with the capture set C . If the unbox operation is part of a closure with body t , then C will contribute to the captured variables $\text{cv}(t)$ of the body and therefore to the capture set of the closure as a whole. In short, unbox operations propagate captures to enclosing closures (whereas, dually, box operations suppress propagation).

To see why this matters, assume for the moment a system with type polymorphism but without boxes, where type variables can range over capturing types but type variables are not themselves tracked in capture sets. Then the following is possible:

```
val framework
  : [X <: Any^{cap} ] -> (x: X) -> (X -> Unit) -> Unit =
  = [X <: Any^{cap} ] => (x: X) => (plugin: X -> Unit) => plugin(x)
```

The framework function combines the two sides of an interaction, with an argument x and an argument plugin acting on x . The interaction is generic over type variable X . Now instantiate framework like this:

```
val c: File^{cap}
val inst
  : (File^{c} -> Unit) -> Unit
  = framework[File^{c}](c)
```

This looks suspicious since `inst` now has a pure type with empty capture set, yet invoking it can access the `c` capability. Here is an example of such an access:

```
val writer
  : File^{c} -> Unit
  = (x: File^{c}) => x.write
inst(writer)
```

This invocation clearly executes an effect on the formal parameter x , which gets instantiated with `c`. Yet both `inst` and `writer` have pure types with no retained capabilities. Note that `writer` gets the necessary capability $\{c\}$ from its argument, so the function itself does not retain capabilities in its environment, which makes it pure. It is difficult to see how a system along these lines could be sound. At the very least, it would violate the capture prediction Lemma 4.12.

Boxing the bound of X and adding the required box/unbox operations rejects the unsound instantiation. The definitions of `framework` and `inst` now become:

```
val framework
  : [X <: □ Any^{cap} ] -> (x: X) -> (X -> Unit) -> Unit =
  = [X <: □ Any^{cap} ] => (x: X) => (plugin: X -> Unit) => plugin(x)

val inst
  : (□ File^{c} -> Unit) -> Unit
```

```
= framework[□ File^{c}](□ c)
```

Now any attempt to invoke `inst` as before would lead to an error:

```
val writer
  : (□ File^{c}) ->{c} Unit
  = (x: File^{c}) => ({c} ◦ x).write
inst(writer) // error
```

Indeed, `writer`, the argument to `inst`, now has the type

```
(□ File^{c}) ->{c} Unit
```

because the `unbox` operation in the lambda's body charges the closure with the capture set `{c}`. Therefore, the argument is now incompatible with `plugin`'s formal parameter type

```
(□ File^{c}) -> Unit
```

which is a pure function type.

This example shows why one cannot simply drop boxes and keep everything else unchanged. But one could also think of several other possibilities.

One alternative is to drop boxes but keep the stratification of shape types and full types. Type variables would still be full types but not shape types. Such a system would certainly be simpler, but it would also be too restrictive to be useful in practice. For instance, it would not be possible to invoke a polymorphic constructor that creates a list of functions that capture some capability.

Another alternative is to drop both boxes and the stratification of shape types and full types. In this case, to regain soundness, one would have to admit capture sets that range over both term variables and type variables. We have explored this alternative system elsewhere [Boruch-Gruszecki et al. 2021]. This alternative turns out to lead to much larger capture sets in practice, since most polymorphic type variables would end up in capture sets. For instance, the classical list `cons` function has a pure type in the boxed system:

```
def cons: [T <: □ Any^{cap}] -> T -> List[T] -> List[T]
```

But in the system that tracks type variables in capture sets, it would have the following more verbose type:

```
def cons: [T <: Any^{cap}]]
  -> (x: T)
  ->{T} (xs: List[T])
  ->{x, T} List[T]^{x, xs, T}
```

In summary, a system with boxes turned out to lead to the best ergonomics of expression among the alternatives we considered. The core property of boxes is that unboxing charges the environment with the capture set of the unboxed type and thus allows to correctly recover captured references in a box without having to propagate these captures into the types of polymorphic type constructors. So in a sense, the conclusion is that one can always unbox (as long as the capture set is not the universal one), but it does not come for free.

We show in a separate work [Xu and Odersky 2023] that boxed types and boxing and unboxing operations can be inferred. That work presents an algorithmic type system that inserts boxed type constructors around capturing type arguments and inserts `box` and `unbox` operations as needed in the terms accessing values of these type arguments. As is typical, the algorithmic type system is significantly more complicated than the declarative system in this article.

Syntax

Term $t, s ::= \mathbf{boundary}[S] x \Rightarrow t \mid \dots$
Shape type $S, R ::= \mathbf{Break}[S] \mid \dots$

Definitions

$\mathbf{cv}(\mathbf{boundary}[S] x \Rightarrow t) = \mathbf{cv}(t) \setminus x$

Subtyping

$$\frac{\Gamma \vdash S_2 <: S_1}{\Gamma \vdash \mathbf{Break}[S_1] <: \mathbf{Break}[S_2]} \quad (\mathbf{BREAK})$$

Typing

$$\frac{\Gamma, x : \mathbf{Break}[S]^\wedge \{\mathbf{cap}\} \vdash t : S \quad x \notin \mathbf{fv}(S)}{\Gamma \vdash \mathbf{boundary}[S] x \Rightarrow t : S} \quad (\mathbf{BOUNDARY}) \qquad \frac{\Gamma \vdash x : \mathbf{Break}[S]^\wedge \{\mathbf{cap}\} \quad \Gamma \vdash y : S}{\Gamma \vdash x y : T} \quad (\mathbf{INVOKE})$$

Fig. 7. Scoped capability extensions to the static rules of System $\mathbf{CC}_{<:\square}$.

One can also turn that around. If we have a sound system with type variables (i.e., by inserting implicit boxed types and box/unbox operation in the way our implementation works), then it is possible to define box and unbox as library operations in the language, along the following lines:

```
class Box[T](elem: T)
def box[T](x: T): Box[T] = new Box[T]
def unbox[T](x: Box[T]): T = x.elem
```

This construction demonstrates that in essence, boxes can be seen as a mechanism to obtain sound polymorphism for capturing types. Once we have a such a system, the functionality of source boxes can also be obtained by defining a parametric class (or an equivalent Church encoding) with a constructor/destructor pair. That is why our implemented language does not need to expose boxed types and primitive box and unbox operations in the source code—a construction like the preceding one is enough to simulate this functionality.

7 SCOPED CAPABILITIES

In this section, we discuss how boxes can be used to ensure that capabilities are well-scoped, based on an extension to $\mathbf{CC}_{<:\square}$. Figure 7 shows the extensions to the static semantics. The extension is minimal: we add a **boundary** form $\mathbf{boundary}[S] x \Rightarrow t$, mirroring a Scala 3 feature [Scala 2022a]. The boundary form delimits a scope that can be broken out of by using the *break capability* $x : \mathbf{Break}[S]$; the form is parameterized by a type argument S that can be inferred in the implementation. A boundary is a more expressive version of a labeled block that can be returned from: it also allows returning across closure and function boundaries since the break capability is a first-class value that can be closed over and passed as an argument. The type system should disallow invoking the capability once the boundary is left, since intuitively at that point there is no scope to be broken out of. As we explained in Section 4.2, a variable x of boxed type can only be opened via an unbox form $C \circ- x$ such that C accounts for the capability in the box. Our plan is simple: we (1) ensure that all capabilities leaving the **boundary** scope are boxed and (2) ensure that the scoped capability cannot be accounted for by any variable other than itself. In this extension, the only way for a scoped capability to leak is by being directly returned from its scope, so it suffices to require in rule (BOUNDARY) that the result of a **boundary** form is pure. To illustrate, consider the following attempt to leak a scoped capability by returning a closure (where

Syntax

Label	l	$::=$	$@123 \mid @456 \mid \dots$
Value	v, w	$::=$	$l_S \mid \dots$
Term	t, s	$::=$	$\mathbf{scope}_{l_S} t \mid \dots$
Captured Reference	p	$::=$	$x \mid l$
Capture Set	C	$::=$	$\{p_1, \dots, p_n\}$

Definitions

$$\mathbf{cv}(\mathbf{scope}_{l_S} t) = \mathbf{cv}(t)$$

$$\mathbf{cv}(l_S) = \{l\}$$

Subcapturing

Base subcapturing rules use p instead of x .

$$\Gamma \vdash \{l\} <: \{\mathbf{cap}\}$$

(SC-LABEL)

Typing

$$\Gamma \vdash l_S : \mathbf{Break}[S]^\wedge \{l\}$$

(LABEL)

$$\frac{\Gamma \vdash t : S}{\Gamma \vdash \mathbf{scope}_{l_S} t : S}$$

(SCOPE)

Evaluation

$$\begin{aligned} \sigma[e[\mathbf{boundary}[S] x \Rightarrow t]] &\longrightarrow \sigma[\mathbf{let} x = l_S \mathbf{in} e[\mathbf{scope}_{l_S} t]] && \text{if } l \text{ fresh} && (\text{ENTER}) \\ \sigma[e_1[\mathbf{scope}_{l_S} e_2[x y]]] &\longrightarrow \sigma[e_1[y]] && \text{if } \sigma(x) = l_S && (\text{BREAKOUT}) \\ \sigma[e[\mathbf{scope}_{l_S} a]] &\longrightarrow \sigma[e[a]] && && (\text{LEAVE}) \end{aligned}$$

$$\text{where Eval context } e ::= \mathbf{scope}_{l_S} [] \mid \dots$$

Fig. 8. Scoped capability extensions to dynamic rules of System $\mathbf{CC}_{<:\Box}$.

$\mathbf{Proc} \triangleq \forall(y : \mathbf{Unit}) \mathbf{Unit}$:

$$\vdash \mathbf{boundary}[\dots] x \Rightarrow \mathbf{let} f = \lambda(y : \mathbf{Unit}) x () \mathbf{in} \Box f : \Box \mathbf{Proc}^\wedge \{\mathbf{cap}\}$$

Since a boundary's result must be pure, we have no choice but to box the closure. Since x is not in scope outside of the boundary, the capture set under the box must be $\{\mathbf{cap}\}$. Since no typing context accounts for $\{\mathbf{cap}\}$, the box cannot be opened anymore and we are safe.

In a fully featured programming language, there are other channels for scoped capabilities to leak (e.g., via mutable state). With boxing, to make such channels sound it suffices to only allow pure values to pass through them. For instance, if we want to store a capability in mutable state, we need to box it; afterward, we can only use it in a typing context that accounts for the capabilities under the box. In more complicated scenarios, a capability may return to its scope after leaving it; such cases could occur, for instance, when we allow sending values between threads and when we allow effect-polymorphic effect handlers [Biernacki et al. 2020; Leijen 2014]. Boxing has been shown to be sound and behave as expected in the latter scenario: the boxed capability can be unboxed once it is back in its scope, but not earlier [Brachthäuser et al. 2022]. Thus, although the extension we show is minimal, it presents all the formal foundations we need for ensuring scoping of capabilities.

7.1 Dynamic Semantics of Scoped Capabilities

Figure 8 shows the extensions to the dynamic semantics of $\mathbf{CC}_{<:\Box}$. We add new evaluation-time term forms; we explain them by inspecting the relevant evaluation rules. Rule (ENTER) reduces a term of the form $\sigma[e[\mathbf{boundary}[S] x \Rightarrow t]]$ to $\sigma[\mathbf{let} x = l_S \mathbf{in} e[\mathbf{scope}_{l_S} t]]$: entering a boundary binds the break capability l_S in the store and adds a *scope* form to the evaluation context. The break capability is a *label* l annotated with the boundary's return type, where a

label represents a boundary's unique runtime identifier. The scope form $\mathbf{scope}_{l_S} t$ is a marker on the stack (formally, the evaluation context), denoting where a boundary ends; all scopes are annotated with their corresponding labels. When the break capability is invoked, the term has the form $\sigma[e_1[\mathbf{scope}_{l_S} e_2[x y]]]$ and the evaluation context is split by a scope form into the part outside and inside the scope. Rule (BREAKOUT) reduces such terms to $\sigma[e_1[y]]$, dropping the scope form together with the inner evaluation context. Once only an answer remains under the scope, rule (LEAVE) reduces $\sigma[e[\mathbf{scope}_{l_S} a]]$ to $\sigma[e[a]]$. Typing ensures that after a boundary is left, its capability is never invoked; otherwise, we could get stuck terms since the scope form needed by (BREAK) would be absent from the evaluation context.

7.2 Metatheory

If we start evaluation from a term well-typed according to the static typing rules (one that does not mention any labels or scope forms), the evaluation rules maintain an invariant: all break capabilities are well-scoped, and all scope labels are unique. Since terms could get stuck without this invariant, we state it formally and incorporate it into the main correctness theorems. For the purposes of our metatheory (including this invariant), we understand labels as primitive capabilities provided by the “runtime” to the program—in particular, the cv of a closed term may now mention labels, which we understand as the primitive capabilities a program can access.

Definition (Captured Variables of Contexts). We extend cv to contexts by $\text{cv}([\]) = \{\}$.

Definition (Proper Program). A term is a *proper program* if it has the form $\sigma[e[t]]$ s.t.:

- for all l such that $l \in \text{cv}(\sigma[e[t]])$:
 - there exists a unique x such that $\sigma(x) = l_S$ for some S
 - there exist unique e_1 and e_2 such that $e = e_1[\mathbf{scope}_{l_S} e_2]$ for the same S
 - for the same e_1 we have $l \notin \text{cv}(e_1)$
- scope forms in $\sigma[e[t]]$ only occur in e .

THEOREM 7.1 (PRESERVATION). *Let $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, where $\sigma[t]$ is a proper program. Then $\sigma[t] \longrightarrow \sigma[t']$ implies that $\Gamma, \Delta \vdash t' : T$ and that $\sigma[t']$ is a proper program.*

THEOREM 7.2 (PROGRESS). *If $\vdash \sigma[t] : T$ and $\sigma[t]$ is a proper program and a proper configuration, then either t is an answer a or $\sigma[t] \longrightarrow \sigma[t']$ for some t' .*

In the base system, we needed Theorem 4.14 to demonstrate that boxes are typed correctly, since unboxing a capability could never lead to a stuck term. In this extension, unboxing an out-of-scope capability *can* lead to a stuck term, so we can demonstrate soundness of the boxing rules in a more direct way, by showing the classical progress and preservation theorems. In fact, Lemma 4.13 and Theorem 7.1 both employ an identical argument in the case for rule (OPEN).

7.2.1 Predicting Used Capabilities. We can understand labels as the primitive capabilities a program may access. This makes the situation more complicated than before: primitive capabilities can now be created and dropped. From the object capability perspective, this is as expected. For example, in Wyvern [Melicher et al. 2017], creating capabilities is a commonplace occurrence, since an object with mutable state counts as a capability. In systems where file handles are capabilities, a capability is created or dropped every time we open or close a file handle.

This means that when reasoning about what capabilities are used, we need to consider what capabilities were created or dropped. To account for this, we reason about *traces*: sets of events that occurred during evaluation.

Definition (Evaluation Trace).

$$\begin{aligned}
 \text{trace}(t_1 \longrightarrow t_2 \longrightarrow \dots \longrightarrow t_n) &= \text{trace}(t_1 \longrightarrow t_2) \cup \text{trace}(t_2 \longrightarrow \dots \longrightarrow t_n) \\
 \text{trace}(\sigma[e[x \ y]] \longrightarrow s) &= \{\text{use}(l)\} \quad \text{if } \sigma(x) = l_S \\
 \text{trace}(\sigma[e[\mathbf{boundary}[S] x \Rightarrow t]] \longrightarrow s) &= \{\text{create}(l)\} \quad \text{if } s = \sigma'[e[\mathbf{scope}_{l_S} t]] \\
 \text{trace}(\sigma[e[\mathbf{scope}_{l_S} a]] \longrightarrow s) &= \{\text{drop}(l)\} \\
 \text{trace}(t_1 \longrightarrow t_2) &= \{\} \quad \text{otherwise}
 \end{aligned}$$

We define the following auxiliary functions.

Definition (Used, Created, and Gained Capabilities).

$$\begin{aligned}
 \text{used}(t \longrightarrow^* s) &= \{x \mid \mathbf{use}(x) \in \text{trace}(t \longrightarrow^* s)\} \\
 \text{created}(t \longrightarrow^* s) &= \{x \mid \mathbf{create}(x) \in \text{trace}(t \longrightarrow^* s)\} \\
 \text{gained}(t \longrightarrow^* s) &= \{x \mid \mathbf{create}(x) \in \text{trace}(t \longrightarrow^* s), \mathbf{drop}(x) \notin \text{trace}(t \longrightarrow^* s)\}
 \end{aligned}$$

The program authority preservation lemma is now stated slightly differently. First, we only consider break capabilities to be primitive. Second, programs can gain authority over new capabilities, but *only* by creating them and *only* until the capabilities are dropped. Typing already ensures that all break capabilities are tracked and labels are always “bound,” so it is now unnecessary to separately define platform contexts and well-typed programs.

LEMMA 7.3 (PROGRAM AUTHORITY PRESERVATION). *Let $t \longrightarrow t'$, where $\vdash t : T$. Then,*

$$\text{cv}(t') \subseteq \text{cv}(t) \cup \text{gained}(t \longrightarrow^* t').$$

Finally, we reformulate the used capability prediction theorem.

THEOREM 7.4 (USED CAPABILITY PREDICTION). *Let $t \longrightarrow^* t'$, where $\vdash t : T$. Then the primitive capabilities used during the evaluation are within the authority of t :*

$$\text{used}(t \longrightarrow^* t') \subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow^* t').$$

8 RELATED WORK

The results presented in this article did not emerge in a vacuum, and many of the underlying ideas appeared individually elsewhere in similar or different form. We follow the structure of the informal presentation in Section 2 and organize the discussion of related work according to the key ideas behind $\text{CC}_{<:\Box}$.

Effects as Capabilities. Establishing effect safety by moderating access to effects via term-level capabilities is not a new idea [Marino and Millstein 2009]. It has been proposed as a strategy to retrofit existing languages with means to reason about effect safety [Choudhury and Krishnaswami 2020; Liu 2016; Osvald et al. 2016]. Recently, it also has been applied as the core principle behind a new programming language featuring effect handlers [Brachthäuser et al. 2020a]. Similar to the preceding prior work, we propose to use term-level capabilities to restrict access to effect operations and other scoped resources with a limited lifetime. Representing effects as capabilities results in a good economy of concepts: existing language features, like term-level binders, can be reused; programmers are not confronted with a completely new concept of effects or regions.

Making Capture Explicit. Having a term-level representation of scoped capabilities introduces the challenge to restrict use of such capabilities to the scope in which they are still live. To address this issue, effect systems have been introduced [Biernacki et al. 2020; Brachthäuser et al. 2020b; Zhang and Myers 2019], but those can result in overly verbose and difficult to understand types [Brachthäuser et al. 2020a]. A third approach, which we follow in this work, is to make capture explicit in the type of functions.

Hannan [1998] proposes a type-based escape analysis with the goal to facilitate stack allocation. The analysis tracks variable reference using a type-and-effect system and annotates every function type with the set of free variables it captures. The authors leave the treatment of effect polymorphism to future work. In a similar spirit, Scherer and Hoffmann [2013] present open closure types to facilitate reasoning about dataflow properties such as non-interference. They present an extension of the simply typed lambda calculus that enhances function types $[\Gamma_0](\tau) \rightarrow \tau$ with the lexical environment Γ_0 that was originally used to type the closure.

Brachthäuser et al. [2022] show System C, which mediates between first- and second-class values with boxes. In their system, scoped capabilities are second-class values. Normally, second-class values cannot be returned from any scope, but in System C they can be boxed and returned from *some* scopes. The type of a boxed second-class value tracks which scoped capabilities it has captured and, accordingly, from which scopes it cannot be returned. System C tracks second-class values with a coeffect-like environment and uses an effect-like discipline for tracking captured capabilities, which can in specific cases be more precise than cv. In comparison, $CC_{<:\Box}$ does not depend on a notion of second-class values and deeply integrates capture sets with subtyping.

Recently, Bao et al. [2021] proposed to qualify types with *reachability sets*. Their *reachability types* allow reasoning about non-interference, scoping, and uniqueness by tracking for each reference what other references it may alias or (indirectly) point to. Their system formalizes subtyping but not universal polymorphism. However, it relates reachability sets along a different dimension than $CC_{<:\Box}$. Whereas in $CC_{<:\Box}$ a subtyping relationship is established between a capability c and the capabilities in the type of c , reachability types assume a subtyping relationship between a variable x and the variable owning the scope where x is defined. Reachability types track detailed points-to and aliasing information in a setting with mutable variables, whereas $CC_{<:\Box}$ is a more foundational calculus for tracking references and capabilities that can be and was used as a guide for an implementation in a complete programming language. It would be interesting to explore how reachability and separation can be tracked in $CC_{<:\Box}$.

Capture Polymorphism. Combining effect tracking with higher-order functions immediately gives rise to effect polymorphism, which has been a long-studied problem.

Similar to the usual (parametric) type polymorphism, the seminal work by Lucassen and Gifford [1988] on type and effect systems featured (parametric) *effect polymorphism* by adding language constructs for explicit region abstraction and application. Similarly, work on region-based memory management [Tofte and Talpin 1997] supports *region polymorphism* by explicit region abstraction and application. Recently, languages with support for algebraic effects and handlers, such as Koka [Leijen 2017] and Frank [Lindley et al. 2017], feature explicit, parametric effect polymorphism.

It has been observed multiple times, for instance, by Osvald et al. [2016] and Brachthäuser et al. [2020a] that parametric effect polymorphism can become verbose and results in complicated types and confusing error messages. Languages sometimes attempt to *hide* the complexity—they “simplify the types more and leave out ‘obvious’ polymorphism” [Leijen 2017]. However, this solution is not satisfying since the full types resurface in error messages. In contrast, we support polymorphism by reusing existing term-level binders and support simplifying types by means of subtyping and subcapturing.

Rytz et al. [2012] present a type-and-effect system in which higher-order functions like `map` can be assigned simple signatures that do not mention effect variables. As in $CC_{<:\Box}$, it is not necessary to modify the signatures of higher-order functions which only call their argument. However, in the “argument-relative” system of Rytz et al., it is impossible to reference an effect of a particular argument. This limits the overall expressivity in their system, compared to $CC_{<:\Box}$ —for instance, it is not possible to type function composition, or in general a function that returns a value whose

effect is relative to its argument. Their system also does not allow user-defined effects, whereas $CC_{<:\Box}$ allows tracking any variable by annotating it with an appropriate capture set.

The problem of how to prevent capabilities from escaping in closures is also addressed by *second-class values* that can only be passed as arguments but not be returned in results or stored in mutable fields. Siek et al. [2012] enforce second-class function arguments using a classical polymorphic effect discipline, whereas Osvald et al. [2016] and Brachthäuser et al. [2020a] present a specialized type discipline for this task. Second-class values cannot be returned or closed-over by first-class functions. However, second-class functions can freely close over capabilities, since they are second class themselves. This gives rise to a convenient and lightweight form of *contextual* effect polymorphism [Brachthäuser et al. 2020a]. Although this approach allows for effect polymorphism with a simple type system, it is also restrictive because it also forbids local returns and retentions of capabilities—a problem solved by adding boxing and unboxing [Brachthäuser et al. 2022].

Foundations of Boxing. Contextual Modal Type Theory (CMTT) [Nanevski et al. 2008] builds on intuitionistic modal logic. In intuitionistic modal logic, the graded propositional constructor $[\Psi]$ A (pronounced *box*) witnesses that A can be proven only using true propositions in Ψ . Judgments in CMTT have two contexts: Γ roughly corresponding to $CC_{<:\Box}$ bindings with $\{\text{cap}\}$ as their capture set, and a modal context Δ roughly corresponding to bindings with concrete capture sets. Bindings in the modal context are necessarily boxed and annotated with a modality $x :: A[\Psi] \in \Delta$. Just like our definition of captured variables in $CC_{<:\Box}$, the definition of free variables by Nanevski et al. [2008] assigns the empty set to a boxed term (i.e., $fv(\text{box}(\Psi.M)) = \{\}$). Similar to our unboxing construct, using a variable bound in the modal context requires that the current context satisfies the modality Ψ , mediated by a substitution σ . Different to CMTT, $CC_{<:\Box}$ does not introduce a separate modal context. It also does not annotate modalities on binders—instead, these are kept in the types. Also different from CMTT, in $CC_{<:\Box}$ unboxing is annotated with a capture set and not a substitution.

Comonadic type systems were introduced to support reasoning about *purity* in existing, impure languages [Choudhury and Krishnaswami 2020]. Very similar to the box modality of CMTT, a type constructor ‘Safe’ witnesses the fact that its values are constructed without using any impure capabilities. The type system presented by Choudhury and Krishnaswami [2020] only supports a binary distinction between *pure* values and *impure* values; however, the authors comment that it might be possible to generalize their system to graded modalities.

In the present work, we use boxing as a practical tool, necessary to obtain concise types when combining capture tracking with parametric type polymorphism.

Coeffect Systems. *Coeffect systems* also attach additional information to bindings in the environment, leading to a typing judgment of the form $\Gamma @ C \vdash e : \tau$. Such systems can be seen as similar in spirit to $CC_{<:\Box}$, where additional information is available about each variable in the environment through the capture set of its type. Petricek et al. [2014] show a general coeffect framework that can be instantiated to track various concepts such as bounded reuse of variables, implicit parameters, and data access. This framework is based on simply typed lambda calculus, and its function types are always coeffect-monomorphic. In contrast, $CC_{<:\Box}$ is based on System $F_{<}$ (thus supporting type polymorphism and subtyping) and supports capture-polymorphic functions.

Object Capabilities. The (object) capability model of programming [Boyland et al. 2001; Crary et al. 1999; Miller 2006] controls security-critical operations by requiring access to a capability. Such a capability can be seen as the constructive proof that the holder is entitled to perform the critical operation. Reasoning about which operations a module can perform is reduced to reasoning about which references to capabilities a module holds.

The Newspeak language [Bracha et al. 2010] features object capabilities. In particular, it features the *platform capability*, an object that grants access to the underlying platform and allows resolving modules and capabilities. The platform capability is similar to the root capability **cap**: a $CC_{<:\Box}$ value whose capture set is $\{\mathbf{cap}\}$ has the authority to access arbitrary capabilities, whereas capturing the Newspeak platform capability grants access to the entire platform.

The Wyvern language [Melicher et al. 2017] implements the object capability model by distinguishing between stateful *resource modules* and *pure modules*. Access to resource modules is restricted and only possible through capabilities. Determining the authority granted by a module amounts to manually inspecting its type signature and all of the type signatures of its transitive imports. To support this analysis, Melicher [2020] extends the language with a fine-grained effect system that tracks access of capabilities in the type of methods.

Figuroa et al. [2016] show an intricately engineered encoding of object capabilities in Haskell. A monad transformer’s private operations can only be called from modules with the appropriate authority. The capabilities may be part of a hierarchy—for example, the ReadWrite capability may subsume the Read and Write capabilities. Capabilities may be shared between modules through encoded friend declarations, and one may determine a module’s authority like in Wyvern.

In $CC_{<:\Box}$, one can statically reason about authority and capabilities simply by inspecting capture sets of types. Additionally, subcapturing naturally allows defining capability hierarchies. If we model modules via function abstraction, the function’s capture set directly reflects its authority. Importantly, $CC_{<:\Box}$ does not include an effect system and thus tracks mention rather than use.

9 CONCLUSION

We introduced a new type system $CC_{<:\Box}$ to track captured references of values. Tracked references are restricted to capabilities, where capabilities are references bootstrapped from other capabilities, starting with the universal capability. Implementing this simple principle then naturally suggests a chain of design decisions:

- (1) Because capabilities are variables, every function must have its type annotated with its free capability variables.
- (2) To manage the scoping of those free variables, function types must be dependently-typed.
- (3) To prevent non-variable terms from occurring in types, the programming language is formulated in MNF.
- (4) Because of type dependency, the let-bindings of MNF have to satisfy the avoidance property, to prevent out-of-scope variables from occurring in types.
- (5) To make avoidance possible, the language needs a rich notion of subtyping on the capture sets.
- (6) Because the capture sets represent object capabilities, the subcapture relation cannot just be the subset relation on sets of variables—it also has to take into account the types of the variables, since the variables may be bound to values which themselves capture capabilities.
- (7) To keep the size of the capture sets from ballooning out of control, the article introduces a box connective with box and unbox rules to control when free variables are counted as visible.

We showed that the resulting system can be used as the basis for lightweight polymorphic effect checking, without the need for effect quantifiers. We also identified three key principles that keep notational overhead for capture tracking low:

- Variables are tracked only if their types have non-empty capture sets. In practice, the majority of variables are untracked and thus do not need to be mentioned at all.

- Subcapturing, subtyping, and subsumption mean that more detailed capture sets can be subsumed by coarser ones.
- Boxed types stop propagation of capture information in enclosing types, which avoids repetition in capture annotations to a large degree.

Our experience so far indicates that the presented calculus is simple and expressive enough to be used as a basis for more advanced effect and resource checking systems and their practical implementations.

APPENDICES

A PROOFS

A.1 Proof Devices

[NOTE: The article should already define well-formedness of types.]

We extend type well-formedness to environments.

Well-Formed Environment

$$\boxed{\vdash \Gamma \text{ wf}}$$

$$\frac{\vdash \Gamma \text{ wf} \quad \Gamma \vdash T \text{ wf}}{\vdash \Gamma, x : T \text{ wf}}$$

$$\frac{\vdash \Gamma \text{ wf} \quad \Gamma \vdash T \text{ wf}}{\vdash \Gamma, X <: T \text{ wf}}$$

$$\vdash \emptyset \text{ wf}$$

To prove Preservation (Theorem A.42), we relate the typing derivation of a term of the form $\sigma[t]$ to the typing derivation for the *plug* term t inside the store σ . We do so with the following definition.

Matching Environment

$$\boxed{\Gamma \vdash \sigma \sim \Delta}$$

$$\frac{\Gamma, x : T \vdash \sigma \sim \Delta \quad \Gamma \vdash v : T \quad x \notin \text{fv}(T)}{\Gamma \vdash \text{let } x = v \text{ in } \sigma \sim x : T, \Delta}$$

$$\Gamma \vdash [] \sim$$

Definition A.1 (Evaluation Context Typing ($\Gamma \vdash e : U \Rightarrow T$)). We say that e can be typed as $U \Rightarrow T$ in Γ iff for all t such that $\Gamma \vdash t : U$, we have $\Gamma \vdash e[t] : T$.

FACT A.2. *If $\sigma[t]$ is a well-typed term in Γ , then there exists a Δ matching σ (i.e., such that $\Gamma \vdash \sigma \sim \Delta$), finding it is decidable, and Γ, Δ is well-formed.*

FACT A.3. *The analogous holds for $e[t]$.*

A.2 Properties of Evaluation Contexts and Stores

In the proof, we use the following metavariables: C, D for capture sets, R, S for shape types, and P, Q, T, U for types.

We also denote the capture set fragment of a type as $\text{cv}(T)$, defined as $\text{cv}(R \wedge C) = C$.

In all our statements, we implicitly assume that all environments are well-formed.

LEMMA A.4 (EVALUATION CONTEXT TYPING INVERSION). $\Gamma \vdash e[s] : T$ implies that for some U we have $\Gamma \vdash e : U \Rightarrow T$ and $\Gamma \vdash s : U$.

PROOF. By induction on the structure of e . If $e = []$, then $\Gamma \vdash s : T$ and clearly $\Gamma \vdash [] : T \Rightarrow T$. Otherwise, $e = \text{let } x = e' \text{ in } t$. Proceed by induction on the typing derivation of $e[s]$. We can only assume that $\Gamma \vdash e[s] : T'$ for some T' s.t. $\Gamma \vdash T' <: T$.

Case (LET). Then, $\Gamma \vdash e'[s] : U'$ and $\Gamma, x : U' \vdash t : T'$ for some U' . By the outer IH, for some U we then have $\Gamma \vdash e' : U \Rightarrow U'$ and $\Gamma \vdash s : U$. The former unfolds to $\forall s'. \Gamma \vdash s' : U \Rightarrow \Gamma \vdash e'[s'] : U'$. We now want to show that $\forall s'. \Gamma \vdash s' : U \Rightarrow \Gamma \vdash e[s'] : T'$. We already have $\Gamma \vdash e'[s'] : U'$ and $\Gamma, x : U' \vdash t : T'$, so we can conclude by (LET).

Case (SUB). Then, $\Gamma \vdash e[s] : T''$ and $\Gamma \vdash T'' <: T'$. We can conclude by the inner IH and (TRANS). $\square$

LEMMA A.5 (EVALUATION CONTEXT REIFICATION). *If both $\Gamma \vdash e : U \Rightarrow T$ and $\Gamma \vdash s : U$, then $\Gamma \vdash e[s] : T$.*

PROOF. Immediate from the definition of $\Gamma \vdash e : U \Rightarrow T$. $\square$

LEMMA A.6 (STORE CONTEXT REIFICATION). *If $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, then $\Gamma \vdash \sigma[t] : T$.*

PROOF. By induction on σ .

Case $\sigma = []$. Immediate.

Case $\sigma = \sigma'[\mathbf{let} x = v \mathbf{in}]$. Then, $\Delta = \Delta', x : U$ for some U . Since $x \notin \text{fv}(T)$ as $\Gamma \vdash T \mathbf{wf}$, by (LET), we have that $\Gamma, \Delta' \vdash \mathbf{let} x = v \mathbf{in} t$, and hence by the induction hypothesis for some U , we have that $\Gamma, x : U \vdash \sigma'[t] : T$. The result follows directly. $\square$

The preceding lemma immediately gives us the following.

COROLLARY A.7 (REPLACEMENT OF TERM UNDER A STORE CONTEXT). *If $\Gamma \vdash \sigma[t] : T$ and $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, then for all t' such that $\Gamma, \Delta \vdash t' : T$ we have $\Gamma \vdash \sigma[t'] : T$.*

A.3 Properties of Subcapturing

LEMMA A.8 (TOP CAPTURE SET). *Let $\Gamma \vdash C \mathbf{wf}$. Then, $\Gamma \vdash C <: \{\mathbf{cap}\}$.*

PROOF. By induction on Γ . If Γ is empty, then C is either empty or $\mathbf{cap} \in C$, so we can conclude by (SC-SET) or (SC-ELEM) correspondingly. Otherwise, $\Gamma = \Gamma', x : S^\Delta D$, and since Γ is well-formed, $\Gamma' \vdash D \mathbf{wf}$. By (SC-SET), we can conclude if for all $y \in C$ we have $\Gamma \vdash \{y\} <: \{\mathbf{cap}\}$. If $y = x$, by IH we derive $\Gamma' \vdash D <: \{\mathbf{cap}\}$, which we then weaken to Γ and conclude by (SC-VAR). If $y \neq x$, then $\Gamma' \vdash \{y\} \mathbf{wf}$, so by IH we derive $\Gamma' \vdash \{y\} <: \{\mathbf{cap}\}$ and conclude by weakening. $\square$

COROLLARY A.9 (EFFECTIVELY TOP CAPTURE SET). *Let $\Gamma \vdash C, D \mathbf{wf}$ such that $\mathbf{cap} \in D$. Then we can derive $\Gamma \vdash C <: D$.*

PROOF. We can derive $\Gamma \vdash C <: \{\mathbf{cap}\}$ by Lemma A.8 and then we can conclude by Lemma A.12 and (SC-ELEM). $\square$

LEMMA A.10 (UNIVERSAL CAPABILITY SUBCAPTURING INVERSION). *Let $\Gamma \vdash C <: D$. If $\mathbf{cap} \in C$, then $\mathbf{cap} \in D$.*

PROOF. By induction on subcapturing. Case (SC-ELEM) immediate, case (SC-SET) by repeated IH, case (SC-VAR) contradictory. $\square$

LEMMA A.11 (SUBCAPTURING DISTRIBUTIVITY). *Let $\Gamma \vdash C <: D$. Then for all $x \in C$, we have $\Gamma \vdash \{x\} <: D$.*

PROOF. By inspection of the last subcapturing rule used to derive $C <: D$. All cases are immediate. If the last rule was (SC-SET), we have our goal as premise. Otherwise, we have $C = \{x\}$ and the goal follows directly. $\square$

LEMMA A.12 (SUBCAPTURING TRANSITIVITY). *If $\Gamma \vdash C_1 <: C_2$ and $\Gamma \vdash C_2 <: C_3$, then $\Gamma \vdash C_1 <: C_3$.*

PROOF. By induction on the first derivation.

Case (SC-ELEM). $C_1 = \{x\}$ and $x \in C_2$, so by Lemma A.11 $\Gamma \vdash \{x\} <: C_3$.

Case (SC-VAR). Then $C_1 = \{x\}$ and $x : R \wedge C_4 \in \Gamma$ and $\Gamma \vdash C_4 <: C_2$. By IH, $\Gamma \vdash C_4 <: C_3$ and we can conclude by (SC-VAR).

Case (SC-SET). By repeated IH and (SC-SET). $\square$

LEMMA A.13 (SUBCAPTURING REFLEXIVITY). *If $\Gamma \vdash C \mathbf{wf}$, then $\Gamma \vdash C <: C$.*

PROOF. By (SC-SET) and (SC-ELEM). $\square$

LEMMA A.14 (SUBTYPING IMPLIES SUBCAPTURING). *If $\Gamma \vdash R_1 \wedge C_1 <: R_2 \wedge C_2$, then $\Gamma \vdash C_1 <: C_2$.*

PROOF. By induction on the subtyping derivation. If (CAPT), immediate. If (TRANS), by IH and subcapturing transitivity Lemma A.12. If (REFL), then $C_1 = C_2$ and we can conclude by Lemma A.13. Otherwise, $C_1 = C_2 = \{\}$ and we can conclude by (SC-SET). $\square$

A.3.1 Subtyping Inversion.

FACT A.15. *Both subtyping and subcapturing are transitive.*

PROOF. Subtyping is intrinsically transitive through (TRANS), whereas subcapturing admits transitivity as per Lemma A.12. $\square$

FACT A.16. *Both subtyping and subcapturing are reflexive.*

PROOF. Again, this is an intrinsic property of subtyping by (REFL) and an admissible property of subcapturing per Lemma A.13. $\square$

LEMMA A.17 (SUBTYPING INVERSION: TYPE VARIABLE). *If $\Gamma \vdash U <: X \wedge C$, then U is of the form $X' \wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash X' <: X$.*

PROOF. By induction on the subtyping derivation.

Case (TVAR), (REFL). Follow from reflexivity (A.16).

Case (CAPT). Then we have $U = S \wedge C'$ and $\Gamma \vdash C' <: C$ and $\Gamma \vdash S <: X$.

This relationship is equivalent to $\Gamma \vdash S \wedge \{\} <: X \wedge \{\}$, on which we invoke the IH.

By IH, we have $S \wedge \{\} = Y \wedge \{\}$ and we can conclude with $U = Y \wedge C'$.

Case (TRANS). Then we have $\Gamma \vdash U <: U$ and $\Gamma \vdash U <: X \wedge C$. We proceed by using the IH twice and conclude by transitivity (A.15).

Other rules are impossible. $\square$

LEMMA A.18 (SUBTYPING INVERSION: CAPTURING TYPE). *If $\Gamma \vdash U <: S \wedge C$, then U is of the form $S' \wedge C'$ such that $\Gamma \vdash C' <: C$ and $\Gamma \vdash S' <: S$.*

PROOF. We take note of the fact that subtyping and subcapturing are both transitive (A.15) and reflexive (A.16). The result follows from straightforward induction on the subtyping derivation. $\square$

LEMMA A.19 (SUBTYPING INVERSION: FUNCTION TYPE). *If $\Gamma \vdash U <: (\forall(x : T_1) T_2) \wedge C$, then U either is of the form $X \wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash X <: \forall(x : T_1) T_2$, or U is of the form $(\forall(x : U_1) U_2) \wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash T_1 <: U_1$ and $\Gamma, x : T_1 \vdash U_2 <: T_2$.*

PROOF. By induction on the subtyping derivation.

Case (TVAR). Immediate.

Case (FUN), (REFL). Follow from reflexivity (A.16).

Case (CAPT). Then we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash S <: \forall(x : T_1) T_2$.

This relationship is equivalent to $\Gamma \vdash S^\wedge \{ \} <: (\forall(x : T_1) T_2)^\wedge \{ \}$, on which we invoke the IH.

By IH, $S^\wedge \{ \}$ might have two forms. If $S^\wedge \{ \} = X^\wedge \{ \}$, then we can conclude with $U = X^\wedge C'$.

Otherwise, we have $S^\wedge \{ \} = (\forall(x : U_1) U_2)^\wedge \{ \}$ and $\Gamma \vdash T_1 <: U_1$ and $\Gamma, x : T_1 \vdash U_2 <: T_2$. Then, $U = (\forall(x : U_1) U_2)^\wedge C'$ lets us conclude.

Case (TRANS). Then we have $\Gamma \vdash U <: U'$ and $\Gamma \vdash U <: (\forall(x : T_1) T_2)^\wedge C$. By IH, U may have one of two forms. If $U = X^\wedge C'$, we proceed with Lemma A.17 and conclude by transitivity (A.15).

Otherwise, $U = (\forall(x : U_1) U_2)^\wedge C'$ and we use the IH again on $\Gamma \vdash U' <: (\forall(x : U_1) U_2)^\wedge C'$. If $U = X^\wedge C''$, we again can conclude by (A.15). Otherwise, if $U = (\forall(x : U_1) U_2)^\wedge C''$, the IH only gives us $\Gamma, x : U_1 \vdash U_2 <: U_2$, which we need to narrow to $\Gamma, x : T_1$ before we can similarly conclude by transitivity (A.15).

Other rules are not possible. $\square$

LEMMA A.20 (SUBTYPING INVERSION: TYPE FUNCTION TYPE). *If $\Gamma \vdash U <: (\forall[X <: S] T)^\wedge C$, then U either is of the form $X^\wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash X <: \forall[X <: S] T$, or U is of the form $(\forall[X <: R] U')^\wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash T <: U'$ and $\Gamma, X <: T \vdash R <: S$.*

PROOF. Analogous to the proof of Lemma A.19. $\square$

LEMMA A.21 (SUBTYPING INVERSION: BOXED TYPE). *If $\Gamma \vdash U <: (\Box T)^\wedge C$, then U either is of the form $X^\wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash X <: \Box T$, or U is of the form $(\Box U')^\wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash U' <: T$.*

PROOF. Analogous to the proof of Lemma A.19. $\square$

A.3.2 Permutation, Weakening, Narrowing.

LEMMA A.22 (PERMUTATION). *Permutating the bindings in the environment up to preserving environment well-formedness also preserves type well-formedness, subcapturing, subtyping, and typing.*

Let Γ and Δ be the original and permuted context, respectively. Then:

- (1) *If $\Gamma \vdash T \mathbf{wf}$, then $\Delta \vdash T \mathbf{wf}$.*
- (2) *If $\Gamma \vdash C_1 <: C_2$, then $\Delta \vdash C_1 <: C_2$.*
- (3) *If $\Gamma \vdash U <: T$, then $\Delta \vdash U <: T$.*
- (4) *If $\Gamma \vdash t : T$, then $\Delta \vdash t : T$.*

PROOF. As usual, order of the bindings in the environment is not used in any rule. $\square$

[NOTE: In fact, arbitrary permutation preserves all the preceding judgments, but it might violate environment well-formedness, and we never want to do that.]

LEMMA A.23 (WEAKENING). *Adding a binding to the environment such that the resulting environment is well-formed preserves type well-formedness, subcapturing, subtyping, and typing.*

Let Γ and Δ be the original and extended context, respectively. Then:

- (1) *If $\Gamma \vdash T \mathbf{wf}$, then $\Delta \vdash T \mathbf{wf}$.*
- (2) *If $\Gamma \vdash C_1 <: C_2$, then $\Delta \vdash C_1 <: C_2$.*
- (3) *If $\Gamma \vdash U <: T$, then $\Delta \vdash U <: T$.*

(4) If $\Gamma \vdash t : T$, then $\Delta \vdash t : T$.

PROOF. As usual, the rules only check if a variable is bound in the environment and all versions of the lemma are provable by straightforward induction. For rules which extend the environment, such as (ABS), we need permutation. All cases are analogous, so we will illustrate only one.

Case (ABS). WLOG, we assume that $\Delta = \Gamma, x : T$. We know that $\Gamma \vdash \lambda(y : U) t' : \forall(y : U) U$, and from the premise of (ABS) we also know that $\Gamma, y : U \vdash t' : U$.

By IH, we have $\Gamma, y : U, x : T \vdash t' : U$. $\Gamma, x : T, y : U$ is still a well-formed environment (as T cannot mention y), and by permutation we have $\Gamma, x : T, y : U \vdash t' : U$. Then by (ABS), we have $\Gamma, x : T \vdash \lambda(y : U) t' : \forall(y : U) U$, which concludes. $\square$

LEMMA A.24 (TYPE BINDING NARROWING). (1) If $\Gamma \vdash S' <: S$ and $\Gamma, X <: S, \Delta \vdash T \mathbf{wf}$, then $\Gamma, X <: S', \Delta \vdash T \mathbf{wf}$.

(2) If $\Gamma \vdash S' <: S$ and $\Gamma, X <: S, \Delta \vdash C_1 <: C_2$, then $\Gamma, X <: S', \Delta \vdash C_1 <: C_2$.

(3) If $\Gamma \vdash S' <: S$ and $\Gamma, X <: S, \Delta \vdash T_1 <: T_2$, then $\Gamma, X <: S', \Delta \vdash T_1 <: T_2$.

(4) If $\Gamma \vdash S' <: S$ and $\Gamma, X <: S, \Delta \vdash t : T$, then $\Gamma, X <: S', \Delta \vdash t : T$.

PROOF. By straightforward induction on the derivations. Only subtyping considers types to which type variables are bound, and the only rule to do so is (TVAR), which we prove in the following. All other cases follow from IH or other narrowing lemmas.

Case (TVAR). We need to prove $\Gamma, X <: S', \Delta \vdash X <: S$, which follows from weakening the lemma premise and using (TRANS) together with (TVAR). $\square$

LEMMA A.25 (TERM BINDING NARROWING). (1) If $\Gamma \vdash U' <: U$ and $\Gamma, x : U, \Delta \vdash T \mathbf{wf}$, then $\Gamma, x : U', \Delta \vdash T \mathbf{wf}$.

(2) If $\Gamma \vdash U' <: U$ and $\Gamma, x : U, \Delta \vdash C_1 <: C_2$, then $\Gamma, x : U', \Delta \vdash C_1 <: C_2$.

(3) If $\Gamma \vdash U' <: U$ and $\Gamma, x : U, \Delta \vdash T_1 <: T_2$, then $\Gamma, x : U', \Delta \vdash T_1 <: T_2$.

(4) If $\Gamma \vdash U' <: U$ and $\Gamma, x : U, \Delta \vdash t : T$, then $\Gamma, x : U', \Delta \vdash t : T$.

PROOF. By straightforward induction on the derivations. Only subcapturing and typing consider types to which term variables are bound. Only (SC-VAR) and (VAR) do so, which we prove in the following. All other cases follow from IH or other narrowing lemmas.

Case (VAR). We know that $U = R \wedge C$ and $\Gamma, x : R \wedge C, \Delta \vdash x : R \wedge \{x\}$. As $\Gamma \vdash U' <: U$, from Lemma A.18 we know that $U' = R' \wedge C'$ and that $\Gamma \vdash R' <: R$. We need to prove that $\Gamma, x : R' \wedge C', \Delta \vdash x : R \wedge \{x\}$. We can do so through (VAR), (SUB), (CAPT), (SC-ELEM) and weakening $\Gamma \vdash R' <: R$.

Case (SC-VAR). Then we know that $C_1 = \{y\}$ and that $y : T \in \Gamma, x : U, \Delta$ and that $\Gamma, x : U, \Delta \vdash \text{cv}(T) <: C_2$.

If $y \neq x$, we can conclude by IH and (SC-VAR).

Otherwise, we have $T = U$. From Lemma A.18 we know that $\Gamma \vdash \text{cv}(U') <: \text{cv}(U)$, and from IH we know that $\Gamma, x : U', \Delta \vdash \text{cv}(U) <: C_2$. By (SC-VAR), to conclude it is enough to have $\Gamma, x : U', \Delta \vdash \text{cv}(U') <: C_2$, which we do have by connecting two previous conclusions by weakening and Lemma A.12. $\square$

A.4 Substitution

A.4.1 *Term Substitution.* We will make use of the following fact.

FACT A.26. If $x : T \in \Gamma$ and $\vdash \Gamma \mathbf{wf}$, then $\Gamma = \Delta_1, x : T, \Delta_2$ and $\Delta_1 \vdash T \mathbf{wf}$ and so $x \notin \text{fv}(T)$.

LEMMA A.27 (TERM SUBSTITUTION PRESERVES SUBCAPTURING). If $\Gamma, x : P, \Delta \vdash C_1 <: C_2$ and $\Gamma \vdash D <: \text{cv}(P)$, then $\Gamma, [x := D]\Delta \vdash [x := D]C_1 <: [x := D]C_2$.

PROOF. Define $\theta \triangleq [x := D]$. By induction on the subcapturing derivation.

Case (SC-ELEM). Then $C_1 = \{y\}$ and $y \in C_2$. Inspect if $y = x$. If no, then our goal is $\Gamma, \theta\Delta \vdash \{y\} <: \theta C_2$. In this case, $y \in \theta C_2$, which lets us conclude by (SC-ELEM). Otherwise, we have $\theta C_2 = (C_2 \setminus \{x\}) \cup D$, as $x \in C_2$. Then our goal is $\Gamma, \theta\Delta \vdash D <: (C_2 \setminus \{x\}) \cup D$, which can be shown by (SC-SET) and (SC-ELEM).

Case (SC-VAR). Then $C_1 = \{y\}$ and $y : S \wedge C_3 \in \Gamma, x : P, \Delta$ and $\Gamma, x : P, \Delta \vdash C_3 <: C_2$.

Inspect if $y = x$. If yes, then our goal is $\Gamma, \theta\Delta \vdash D <: \theta C_2$. By IH, we know that $\Gamma, \theta\Delta \vdash \theta C_3 <: \theta C_2$. As $x = y$, we have $P = S \wedge C_3$, and therefore based on an initial premise of the lemma, we have $\Gamma \vdash D <: C_3$. Then by weakening and IH, we know that $\Gamma, \theta\Delta \vdash \theta D <: \theta C_3$, which means we can conclude by Lemma A.12.

Otherwise, $x \neq y$, and our goal is $\Gamma, \theta\Delta \vdash C_1 <: \theta C_2$. We inspect where y is bound.

Case $y \in \text{dom}(\Gamma)$. Then note that $y \notin C_3$ by Fact A.26. By IH, we have $\Gamma, \theta\Delta \vdash \theta C_3 <: \theta C_2$. We can conclude by (SC-VAR) as $[x := D]C_3 = C_3$ and $y : P \wedge C_3 \in \Gamma, \theta\Delta$.

Case $y \in \text{dom}(\Delta)$. Then $y : \theta(P \wedge C_3) \in \Gamma, \theta\Delta$ and we can conclude by IH and (SC-VAR).

Case (SC-SET). Then $C_1 = \{y_1, \dots, y_n\}$ and we inspect if $x \in C_1$.

If not, then for all $y \in C_1$ we have $\theta\{y\} = \{y\}$, and so we can conclude by repeated IH on our premises and (SC-SET).

If yes, then we know that $\forall y \in C_1. \Gamma, x : P, \Delta \vdash \{y\} <: C_2$. We need to show that $\Gamma, \theta\Delta \vdash \theta C_1 <: \theta C_2$. By (SC-SET), it is enough to show that if $y' \in \theta C_1$, then $\Gamma, \theta\Delta \vdash \{y'\} <: \theta C_2$. For each such y' , there exists $y \in C_1$ such that $y' \in \theta\{y\}$. For this y , from a premise of (SC-SET) we know that $\Gamma, x : P, \Delta \vdash \{y\} <: \theta C_2$ and so by IH we have $\Gamma, \theta\Delta \vdash \theta\{y\} <: \theta C_2$. Based on that, by Lemma A.12 we also have $\Gamma, \theta\Delta \vdash \{y'\} <: \theta C_2$, which is our goal. $\square$

LEMMA A.28 (TERM SUBSTITUTION PRESERVES SUBTYPING). *If $\Gamma, x : P, \Delta \vdash U <: T$ and $\Gamma \vdash y : P$, then $\Gamma, [x := y]\Delta \vdash [x := y]U <: [x := y]T$.*

PROOF. Define $\theta \triangleq [x := y]$. Proceed by induction on the subtyping derivation.

Case (REFL), (TOP). By same rule.

Case (CAPT). By IH and Lemma A.30 and (CAPT).

Case (TRANS), (BOXED), (FUN), (TFUN). By IH and re-application of the same rule.

Case (TVAR). Then $U = Y$ and $T = S$ and $Y <: S \in \Gamma, x : U, \Delta$ and our goal is $\Gamma, \theta\Delta \vdash \theta Y <: \theta S$. Note that $x \neq Y$ and inspect where Y is bound. If $Y \in \text{dom}(\Gamma)$, we have $Y <: S \in \Gamma, \theta\Delta$ and since $x \notin \text{fv}(S)$ (Fact A.26), $\theta(S) = S$. Then, we can conclude by (TVAR). Otherwise, if $Y \in \text{dom}(\Delta)$, we have $Y <: \theta S \in \Gamma, \theta\Delta$ and again we can conclude by (TVAR). $\square$

LEMMA A.29 (TERM SUBSTITUTION PRESERVES TYPING). *If $\Gamma, x : P, \Delta \vdash t : T$ and $\Gamma \vdash x' : P$, then $\Gamma, [x := x']\Delta \vdash [x := x']t : [x := x']T$.*

PROOF. Define $\theta \triangleq [x := x']$. Proceed by induction on the typing derivation.

Case (VAR). Then $t = y$ and $y : S \wedge C \in \Gamma, x : P, \Delta$ and $T = S \wedge \{y\}$ and our goal is $\Gamma, \theta\Delta \vdash y : \theta(S \wedge \{y\})$.

If $y = x$, then $P = S \wedge C$ and $\theta(S \wedge \{x\}) = S \wedge \{x'\}$. Our goal is $\Gamma, \theta\Delta \vdash x' : S \wedge \{x'\}$ and we can conclude by (VAR).

Otherwise, $y \neq x$ and we inspect where y is bound.

If $y \in \text{dom}(\Gamma)$, then $x \notin \text{fv}(S \wedge C)$ and so $\theta(S \wedge \{z\}) = S \wedge \{z\}$ and we can conclude by (VAR).

Otherwise, $y \in \text{dom}(\Delta)$, so $y : \theta(S \wedge C) \in \Gamma, \theta\Delta$ and we can conclude by (VAR).

Case (SUB). By IH, Lemma A.28 and (SUB).

Case (ABS). Then $t = \lambda(y : Q). t', T = (\forall(y : Q) T')^{\wedge} cv(t)$ and $\Gamma, x : P, \Delta, y : Q \vdash t' : T'$.

By IH, we have that $\Gamma, \theta\Delta, y : \theta Q \vdash \theta t' : \theta T'$. We note that $cv(\theta t) = \theta cv(t)$, which lets us conclude by (ABS).

Case (TABS). Similar to previous rule.

Case (APP). Then $t = z_1 z_2$ and $\Gamma, x : P, \Delta \vdash z_1 : (\forall(y : Q) T')^{\wedge} C$ and $\Gamma, x : P, \Delta \vdash z_2 : Q$ and $T = [y := z_2]T'$.

By IH, we have $\Gamma, \theta\Delta \vdash \theta z_1 : \theta((\forall(y : Q) T')^{\wedge} C)$ and $\Gamma, \theta\Delta \vdash \theta z_2 : \theta Q$.

Then by (APP), we have $\Gamma, \theta\Delta \vdash \theta(z_1 z_2) : [y := \theta z_2]\theta T'$.

As $y \neq x$ and $y \neq x'$, we have $[y := \theta z_2]\theta T' = \theta([y := z_2]T')$, which concludes.

Case (TAPP). Similar to previous rule.

Case (BOX). Then $t = \Box z$ and $\Gamma, x : P, \Delta \vdash z : S^{\wedge} C$ and $T = \Box S^{\wedge} C$.

By IH, we have $\Gamma, \theta\Delta \vdash \theta z : \theta S^{\wedge} \theta C$. If $x \notin C$, we have $\theta C = C$ and $C \subseteq \text{dom}(\Gamma, \theta\Delta)$, which lets us conclude by (BOX). Otherwise, $\theta C = (C \setminus \{x\}) \cup \{y\}$. As $\Gamma \vdash y : U$, $\theta C \subseteq \text{dom}(\Gamma, \theta\Delta)$, which again lets us conclude by (BOX).

Case (UNBOX). Analogous to the previous rule. Note that we just swap the types in the premise and the conclusion.

Case (LET). Then $t = \text{let } y = s \text{ in } t'$ and $\Gamma, x : P, \Delta \vdash s : Q$ and $\Gamma, x : P, \Delta, y : Q \vdash t' : T$. By the IH, we have $\Gamma, \theta\Delta \vdash \theta s : \theta Q$ and $\Gamma, \theta\Delta, y : \theta Q \vdash \theta t' : \theta T$.

Then by (LET), we also have $\Gamma, \theta\Delta \vdash \theta(\text{let } y = s \text{ in } t') : \theta T$, which concludes. $\square$

A.4.2 Type Substitution.

LEMMA A.30 (TYPE SUBSTITUTION PRESERVES SUBCAPTURING). *If $\Gamma, X <: S, \Delta \vdash C <: D$ and $\Gamma \vdash R <: S$, then $\Gamma, [X := R]\Delta \vdash C <: D$.*

PROOF. Define $\theta \triangleq [X := R]$. Proceed by induction on the subcapturing derivation.

Case (SC-SET), (SC-ELEM). By IH and same rule.

Case (SC-VAR). Then $C = \{y\}$, $y : S' \wedge C' \in \Gamma, X <: S, \Delta$, $y \neq X$. Inspect where y is bound. If $y \in \text{dom}(\Gamma)$, we have $y : S' \wedge C' \in \Gamma, \theta\Delta$. Otherwise, by definition of substitution we have $y : \theta S' \wedge C' \in \Gamma, \theta\Delta$. In both cases, we can conclude by (SC-VAR), since y is still bound to a type whose capture set is C' . $\square$

LEMMA A.31 (TYPE SUBSTITUTION PRESERVES SUBTYPING). *If $\Gamma, X <: S, \Delta \vdash U <: T$ and $\Gamma \vdash R <: S$, then $\Gamma, [X := R]\Delta \vdash [X := R]U <: [X := R]T$.*

PROOF. Define $\theta \triangleq [X := R]$. Proceed by induction on the subtyping derivation.

Case (REFL), (TOP). By same rule.

Case (CAPT). By IH and Lemma A.30 and (CAPT).

Case (TRANS), (BOXED), (FUN), (TFUN). By IH and re-application of the same rule.

Case (TVAR). Then $U = Y$ and $T = S'$ and $Y <: S' \in \Gamma, X <: S, \Delta$ and our goal is $\Gamma, X <: S, \Delta \vdash \theta Y <: \theta S'$. If $Y = X$, by lemma premise and weakening. Otherwise, inspect where Y is bound. If $Y \in \text{dom}(\Gamma)$, we have $Y <: S' \in \Gamma, \theta\Delta$ and since $X \notin \text{fv}(S')$ (Fact A.26), $\theta S' = S'$. Then, we can conclude by (TVAR). Otherwise, if $Y \in \text{dom}(\Delta)$, we have $Y <: \theta S' \in \Gamma, \theta\Delta$ and we can conclude by (TVAR). $\square$

LEMMA A.32 (TYPE SUBSTITUTION PRESERVES TYPING). *If $\Gamma, X <: S, \Delta \vdash t : T$ and $\Gamma \vdash R <: S$, then $\Gamma, [X := R]\Delta \vdash [X := R]t : [X := R]T$.*

PROOF. Define $\theta \triangleq [X := R]$. Proceed by induction on the typing derivation.

Case (VAR). Then $t = y$, $y : S' \wedge C \in \Gamma, X <: S, \Delta$, $y \neq X$, and our goal is $\Gamma, \theta\Delta \vdash y : \theta S' \wedge \{y\}$.

Inspect where y is bound. If $y \in \text{dom}(\Gamma)$, then $y : S' \wedge C \in \Gamma, \theta\Delta$ and $X \notin \text{fv}(S')$ (Fact A.26).

Then, $\theta(S' \wedge C) = S' \wedge C$ and we can conclude by (VAR). Otherwise, $y : \theta S' \wedge C \in \Gamma, \theta\Delta$ and we can directly conclude by (VAR).

Case (ABS), (TABS). In both rules, observe that type substitution does not affect cv and conclude by IH and rule re-application.

Case (APP). Then we have $t = x y$ and $\Gamma, X <: S, \Delta \vdash x : (\forall(z : U) T_0) \wedge C$ and $T = [z := y]T_0$.

We observe that $\theta[z := y]T_0 = [z := y]\theta T_0$ and $\theta t = t$ and conclude by IH and (APP).

Case (TAPP). Then we have $t = x[S']$ and $\Gamma, X <: S, \Delta \vdash x : (\forall[Z <: S'] T_0) \wedge C$ and $T = [Z := S']T_0$.

We observe that $\theta[Z := S']T_0 = [Z := \theta S']\theta T_0$. By IH, $\Gamma, \theta\Delta \vdash x : (\forall[Z <: \theta S'] T_0) \wedge C$. Then, we can conclude by (TAPP).

Case (BOX). Then $t = \Box y$ and $\Gamma, X <: S, \Delta \vdash y : S' \wedge C$ and $T = \Box S' \wedge C$, and our goal is $\Gamma, \theta\Delta \vdash y : \Box(S' \wedge C)$.

Inspect where y is bound. If $y \in \text{dom}(\Gamma)$, then $y : S' \wedge C \in \Gamma, \theta\Delta$ and $X \notin \text{fv}(S')$ (Fact A.26). Then, $\theta(S' \wedge C) = S' \wedge C$ and we can conclude by (BOX). Otherwise, $y : \theta S' \wedge C \in \Gamma, \theta\Delta$ and we can directly conclude by (BOX).

Case (UNBOX). Proceed analogously to the case for (BOX)—we just swap the types in the premise and in the consequence.

Case (SUB). By IH and A.31.

Case (LET). Then $t = \text{let } y = s \text{ in } t'$ and $\Gamma, x : P, \Delta \vdash s : Q$ and $\Gamma, x : P, \Delta, y : Q \vdash t' : T$. By the IH, we have $\Gamma, \theta\Delta \vdash \theta s : \theta Q$ and $\Gamma, \theta\Delta, y : \theta Q \vdash \theta t' : \theta T$.

Then by (LET), we also have $\Gamma, \theta\Delta \vdash \theta(\text{let } y = s \text{ in } t') : \theta T$, which concludes. $\square$

A.5 Main Theorems: Soundness

A.5.1 Preliminaries. As we state Preservation (Theorem A.42) in a non-empty environment, we need to show canonical forms lemmas in such an environment as well. To do so, we need to know that values cannot be typed with a type that is a type variable, which normally follows from the environment being empty. Instead, we show the following lemma.

LEMMA A.33 (VALUE TYPING). *If $\Gamma \vdash v : T$, then T is not of the form $X \wedge C$.*

PROOF. By induction on the typing derivation.

For rule (SUB), we know that $\Gamma \vdash v : U$ and $\Gamma \vdash U <: T$. Assuming $T = X \wedge C$, we have a contradiction by Lemma A.17 and IH.

Rules (BOX), (ABS), (TABS) are immediate, and other rules are not possible. $\square$

LEMMA A.34 (CANONICAL FORMS: TERM ABSTRACTION). *If $\Gamma \vdash v : (\forall(x : U) T) \wedge C$, then we have $v = \lambda(x : U') t$ and $\Gamma \vdash U <: U'$ and $\Gamma, x : U \vdash t : T$.*

PROOF. By induction on the typing derivation.

For rule (SUB), we observe that by Lemma A.19 and by Lemma A.33, the subtype is of the form $(\forall(y : U'') T') \wedge C'$ and we have $\Gamma \vdash U <: U''$. By IH, we know that $v = \lambda(x : U') t$ and $\Gamma \vdash U'' <: U'$ and $\Gamma, x : U'' \vdash t : T$. By (TRANS), we have $\Gamma \vdash U <: U'$ and by narrowing we have $\Gamma, x : U \vdash t : T$, which concludes.

Rule (ABS) is immediate, and other rules cannot occur. $\square$

LEMMA A.35 (CANONICAL FORMS: TYPE ABSTRACTION). *If $\Gamma \vdash v : (\forall[X <: S] T) \wedge C$, then we have $v = \lambda[X <: S'] t$ and $\Gamma \vdash S <: S'$ and $\Gamma, X <: S \vdash t : T$.*

PROOF. Analogous to the proof of Lemma A.34. $\square$

LEMMA A.36 (CANONICAL FORMS: BOXED TERM). *If $\Gamma \vdash v : (\Box T) \wedge C$, then $v = \Box x$ and $\Gamma \vdash x : T$.*

PROOF. Analogous to the proof of Lemma A.34. $\square$

LEMMA A.37 (VARIABLE TYPING INVERSION). *If $\Gamma \vdash x : S \wedge C$, then $x : S' \wedge C' \in \Gamma$ and $\Gamma \vdash S' <: S$ and $\Gamma \vdash \{x\} <: C$ for some C' and S' .*

PROOF. By induction on the typing derivation.

Case (SUB). Then $\Gamma \vdash x : S'' \wedge C''$ and $\Gamma \vdash S'' \wedge C'' <: S \wedge C$. By the IH, we have $\Gamma \vdash x : S' \wedge C'$ and $\Gamma \vdash S' <: S''$ and $\Gamma \vdash x <: C''$. Then by Lemma A.18, we have $\Gamma \vdash S'' <: S$ and $\Gamma \vdash C'' <: C$, which lets us conclude by (TRANS) and transitivity of subcapturing.

Case (VAR). Then $\Gamma \vdash x : S \wedge C'$ and $C = \{x\}$. We can conclude with $S' = S$ by (REFL) and reflexivity of subcapturing. $\square$

LEMMA A.38 (VARIABLE LOOKUP INVERSION). *If we have both $\Gamma \vdash \sigma \sim \Delta$ and $x : S \wedge C \in \Gamma, \Delta$, then $\sigma(x) = v$ implies that $\Gamma, \Delta \vdash v : S \wedge C$.*

PROOF. By structural induction on σ . It is not possible for σ to be empty.

Otherwise, $\sigma = \sigma'[\text{let } y = v \text{ in } _]$ and for some U we have both $\Delta = \Delta', y : U$ and $\Gamma, \Delta' \vdash v : U$. [NOTE: We remove bindings from the “inside” to make induction possible in the following.]

If $y \neq x$, we can proceed by IH as x can also be typed in Γ, Δ' , after which we can conclude by weakening. Otherwise, $U = S \wedge C$ and we can conclude by weakening. $\square$

LEMMA A.39 (TERM ABSTRACTION LOOKUP INVERSION). *If $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash x : (\forall(z : U) T) \wedge C$ and $\sigma(x) = \lambda(z : U') t$, then $\Gamma, \Delta \vdash U <: U'$ and $\Gamma, \Delta, z : U \vdash t : T$.*

PROOF. A corollary of Lemma A.38 and Lemma A.34. $\square$

LEMMA A.40 (TYPE ABSTRACTION LOOKUP INVERSION). *If $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash x : (\forall[Z <: U] T) \wedge C$ and $\sigma(x) = \lambda[Z <: U'] t$, then $\Gamma, \Delta \vdash U <: U'$ and $\Gamma, \Delta, Z <: U \vdash t : T$.*

PROOF. A corollary of Lemma A.38 and Lemma A.35. $\square$

LEMMA A.41 (BOX LOOKUP INVERSION). *If $\Gamma \vdash \sigma \sim \Delta$ and $\sigma(x) = \Box y$ and $\Gamma, \Delta \vdash x : \Box T$, then $\Gamma, \Delta \vdash y : T$.*

PROOF. A corollary of Lemma A.38 and Lemma A.36. $\square$

A.5.2 Soundness. In this section, we show the classical soundness theorems.

THEOREM A.42 (PRESERVATION). *If we have $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, then $\sigma[t] \longrightarrow \sigma[t']$ implies that $\Gamma, \Delta \vdash t' : T$.*

PROOF. We proceed by inspecting the rule used to reduce $\sigma[t]$.

Case (APPLY). Then we have $t = e[x y]$ and $\sigma(x) = \lambda(z : U) s$ and $t' = e[z := y]s$.

By Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash x y : Q$. The typing derivation of $x y$ must start with an arbitrary number of (SUB) rules, followed by (APP). We proceed by induction on the number of (SUB) rules. In both base and inductive cases, we can only assume that $\Gamma, \Delta \vdash x y : Q'$ for some Q' such that $\Gamma, \Delta \vdash Q' <: Q$.

In the inductive case, $\Gamma, \Delta \vdash x y : Q'$ is derived by (SUB), so we also have some Q'' such that $\Gamma, \Delta \vdash x y : Q''$ and $\Gamma, \Delta \vdash Q'' <: Q'$. We have $\Gamma, \Delta \vdash Q'' <: Q$ by (TRANS), so we can conclude by using the inductive hypothesis on $\Gamma, \Delta \vdash x y : Q''$.

In the base case, $\Gamma, \Delta \vdash x y : Q'$ is derived by (APP), so for some Q'' we have $\Gamma, \Delta \vdash x : \forall(z : U') Q''$ and $\Gamma, \Delta \vdash y : U'$ and $Q' = [z := y]Q''$. By Lemma A.39, we have $\Gamma, \Delta, z : U' \vdash s : Q''$. By Lemma A.29, we have $\Gamma, \Delta \vdash [z := y]s : [z := y]Q''$, and since $Q' = [z := y]Q''$, by (SUB) we have $\Gamma, \Delta \vdash [z := y]s : Q$.

To conclude that $t' = e[[z := y]s]$ can be typed as T , we use Lemma A.5.

Case (TAPPLY), (OPEN). As above.

Case (RENAME). Then we have $t = e[\mathbf{let} x = y \mathbf{in} s]$ and $t' = e[[x := y]s]$.

Again, by Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \mathbf{let} x = y \mathbf{in} s : Q$.

We again proceed by induction on number of (SUB) rules at the start of the typing derivation for $\mathbf{let} x = y \mathbf{in} s$, again only assuming that we can type the plug as some Q' such that $Q' <: Q$. The inductive case proceeds exactly as before.

In the base case, (LET) was used to derive that $\Gamma, \Delta \vdash \mathbf{let} x = y \mathbf{in} s : Q'$. The premises are $\Gamma, \Delta \vdash y : U$ and $\Gamma, \Delta, x : U \vdash s : Q'$ and $x \notin \text{fv}(Q')$. By Lemma A.29, we have $\Gamma, \Delta \vdash [x := y]s : [x := y]Q'$. Because $x \notin \text{fv}(Q')$, $[x := y]Q' = Q'$, which means that we can again conclude by (SUB) and Lemma A.5.

Case (LIFT). Then we have $t = e[\mathbf{let} x = v \mathbf{in} s]$ and $t' = \mathbf{let} x = v \mathbf{in} e[s]$.

Again, by Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \mathbf{let} x = v \mathbf{in} s : Q$.

We again proceed by induction on number of (SUB) rules at the start of the typing derivation for $\mathbf{let} x = v \mathbf{in} s$, again only assuming that we can type the plug as some Q' such that $Q' <: Q$. The inductive case proceeds exactly as before.

In the base case, (LET) was used to derive that $\Gamma, \Delta \vdash \mathbf{let} x = v \mathbf{in} s : Q'$. The premises are $\Gamma, \Delta \vdash v : U$ and $\Gamma, \Delta, x : U \vdash s : Q'$ and $x \notin \text{fv}(Q')$.

By weakening of typing, we also have $\Gamma, \Delta, x : U \vdash e : Q \Rightarrow T$. Then by (SUB) and Lemma A.5, we have $\Gamma, \Delta, x : U \vdash e[s] : T$. Since $\Gamma, \Delta \vdash T \mathbf{wf}$, by Barendregt $x \notin \text{fv}(T)$, so by (LET) we have $\Gamma, \Delta \vdash \mathbf{let} x = v \mathbf{in} e[s] : T$, which concludes. $\square$

Definition A.43 (Proper Configuration). We say that a term form $\sigma[t]$ is a *canonical configuration* (of the entire term into store context σ and the plug t) if t is not of the form $\mathbf{let} x = v \mathbf{in} t'$.

FACT A.44. Every term has a corresponding proper configuration, and finding it is decidable.

LEMMA A.45 (EXTRACTION OF BOUND VALUE). If $\Gamma, \Delta \vdash x : T$ and $\Gamma \vdash \sigma \sim \Delta$ and $x \in \text{dom}(\Delta)$, then $\sigma(x) = v$.

PROOF. By structural induction on Δ . If Δ is empty, we have a contradiction. Otherwise, $\Delta = \Delta', z : T'$ and $\sigma = \sigma'[\mathbf{let} z = v \mathbf{in}]$ and $\Gamma, \Delta', z : T' \vdash v : T'$. Note that Δ is the environment matching σ and can only contain term bindings. If $z = x$, we can conclude immediately, and otherwise if $z \neq x$, we can conclude by IH. $\square$

THEOREM A.46 (PROGRESS). If $\vdash \sigma[e[t]] : T$ and $\sigma[e[t]]$ is a proper configuration, then either $e[t] = a$, or there exists $\sigma[t']$ such that $\sigma[e[t]] \longrightarrow \sigma[t']$.

PROOF. Since $\sigma[e[t]]$ is well-typed in the empty environment, there clearly must be some Δ such that $\vdash \sigma \sim \Delta$ and $\Delta \vdash e[t] : T$. By Lemma A.4, we have that $\Delta \vdash t : P$ for some P . We proceed by induction on the derivation of this derivation.

Case (VAR). Then $t = x$.

If e is non-empty, $e[x] = e'[\mathbf{let} y = x \mathbf{in} t']$ and we can step by (RENAME); otherwise, immediate.

Case (ABS), (TABS), (BOX). Then $t = v$.

If e is non-empty, $e[v] = e'[\mathbf{let} x = v \mathbf{in} t']$ and we can step by (LIFT); otherwise, immediate.

Case (APP). Then $t = x y$ and $\Delta \vdash x : (\forall(z : U) T_0) \wedge C$ and $\Delta \vdash y : U$.

By Lemmas A.45 and A.34, $\sigma(x) = \lambda(z : U') t'$, which means we can step by (APPLY).

Case (TAPP). Then $t = x [S]$ and $\Delta \vdash x : (\forall[Z <: S] T_0) \wedge C$.

By Lemmas A.45 and A.35, $\sigma(x) = \lambda[z <: S'] t'$, which means we can step by (TAPPLY).

Case (UNBOX). Then $t = C \multimap x$ and $\Delta \vdash x : \Box S^\wedge C$.

By Lemmas A.45 and A.36, $\sigma(x) = \Box y$, which means we can step by (OPEN).

Case (LET). Then $t = \mathbf{let} \ x = s \ \mathbf{in} \ t'$ and we proceed by IH on s , with $e[\mathbf{let} \ x = [] \ \mathbf{in} \ t']$ as the evaluation context.

Case (SUB). By IH. $\square$

A.5.3 Consequences.

LEMMA A.47 (CAPTURE PREDICTION FOR ANSWERS). *If $\Gamma \vdash \sigma[a] : S^\wedge C$, then $\Gamma \vdash \sigma[a] : S^\wedge \text{cv}(\sigma[a])$ and $\Gamma \vdash \text{cv}(\sigma[a]) <: C$.*

PROOF. By induction on the typing derivation.

Case (SUB). Then $\Gamma \vdash \sigma[a] : S' \wedge C'$ and $\Gamma \vdash S' \wedge C' <: S^\wedge C$. By IH, $\Gamma \vdash \sigma[a] : S' \wedge \text{cv}(\sigma[a])$ and $\Gamma \vdash \text{cv}(\sigma[a]) <: C'$. By Lemma A.18, we have that $\Gamma \vdash C' <: C$ and $\Gamma \vdash S' <: S$.

To conclude, we need $\Gamma \vdash \sigma[a] : S^\wedge \text{cv}(\sigma[a])$ and $\Gamma \vdash \text{cv}(\sigma[a]) <: C$, which we respectively have by subsumption and Lemma A.12.

Case (VAR), (ABS), (TABS), (BOX). Then σ is empty and $C = \text{cv}(a)$. One goal is immediate, the other follows from Lemma A.13.

Case (LET). Then $\sigma = \mathbf{let} \ x = v \ \mathbf{in} \ \sigma'$ and $\Gamma, x : U \vdash \sigma'[a] : S^\wedge C$ and $x \notin C$.

By IH, $\Gamma, x : U \vdash \sigma'[a] : S^\wedge \text{cv}(\sigma'[a])$ and $\Gamma, x : U \vdash \text{cv}(\sigma'[a]) <: C$.

By Lemma A.27, we have $\Gamma \vdash [x := \text{cv}(v)](\text{cv}(\sigma'[a])) <: [x := \text{cv}(v)]C$.

By definition, $[x := \text{cv}(v)](\text{cv}(\sigma'[a])) = \text{cv}(\mathbf{let} \ x = v \ \mathbf{in} \ \sigma'[a])$, and we also already know that $x \notin C$.

This lets us conclude, as we have $\Gamma \vdash \text{cv}(\mathbf{let} \ x = v \ \mathbf{in} \ \sigma'[a]) <: C$.

Other rules cannot occur. $\square$

LEMMA A.48 (CAPTURE PREDICTION FOR TERMS). *Let $\vdash \sigma \sim \Delta$ and $\Delta \vdash t : S^\wedge C$. Then $e[t] \longrightarrow^* e[\sigma'[a]]$ implies that $\Delta \vdash \text{cv}(\sigma'[a]) <: C$.*

PROOF. By preservation, $\vdash \sigma'[a] : S^\wedge C$, which lets us conclude by Lemma A.47. $\square$

A.6 Correctness of Boxing

A.6.1 *Relating cv and Stores.* We want to relate the cv of a term of the form $\sigma[t]$ with $\text{cv}(t)$ such that for some definition of ‘resolve’, we have

$$\text{cv}(\sigma[t]) = \text{resolve}(\sigma, \text{cv}(t)).$$

Let us consider term of the form $\sigma[t]$ and a store σ of the form $\mathbf{let} \ x = v \ \mathbf{in} \ \sigma'$. There are two rules that could be used to calculate $\text{cv}(\mathbf{let} \ x = v \ \mathbf{in} \ \sigma')$:

$$\begin{aligned} \text{cv}(\mathbf{let} \ x = v \ \mathbf{in} \ t) &= \text{cv}(t) && \text{if } x \notin \text{cv}(t) \\ \text{cv}(\mathbf{let} \ x = s \ \mathbf{in} \ t) &= \text{cv}(s) \cup \text{cv}(t) \setminus x. \end{aligned}$$

Observe that since we know that x is bound to a value, we can reformulate these rules as

$$\text{cv}(\mathbf{let} \ x = v \ \mathbf{in} \ t) = [x := \text{cv}(v)] \text{cv}(t),$$

which means that we should be able to define ‘resolve’ with a substitution. We will call this substitution a *store resolver*, and we define it as

$$\begin{aligned} \text{resolver}(\mathbf{let} \ x = v \ \mathbf{in} \ \sigma) &= [x := \text{cv}(v)] \circ \text{resolver}(\sigma) \\ \text{resolver}([]) &= \text{id}. \end{aligned}$$

Importantly, note that we use *composition* of substitutions. We have

$$\text{resolver}(\mathbf{let } x = a \mathbf{ in let } y = x \mathbf{ in } []) \equiv [x := \{a\}, y := \{a\}].$$

With the above, we define *resolve* as

$$\text{resolve}(\sigma, C) = \text{resolver}(\sigma)(C).$$

This definition satisfies our original desired equality with *cv*.

FACT A.49. *For all terms t of the form $\sigma[s]$, we have $\text{cv}(t) = \text{resolve}(\sigma, \text{cv}(s))$*

A.6.2 Relating *cv* and Evaluation Contexts. We now relate *cv* to evaluation contexts e . First, note that by definition of *cv* we have the following.

FACT A.50. *For all terms t of the form $\mathbf{let } x = s \mathbf{ in } t'$ such that s is not a value, we have $\text{cv}(t) = \text{cv}(s) \cup \text{cv}(t') \setminus x$.*

Accordingly, we extend *cv* to evaluation contexts ($\text{cv}(e)$) as follows:

$$\begin{aligned} \text{cv}(\mathbf{let } x = e \mathbf{ in } t) &= \text{cv}(e) \cup \text{cv}(t) \setminus x \\ \text{cv}([]) &= \{\}. \end{aligned}$$

We then have the following.

FACT A.51. *For all terms t of the form $e[s]$ such that s is not a value, we have $\text{cv}(t) = \text{cv}(e) \cup \text{cv}(s)$.*

A.6.3 Relating *cv* to Store and Evaluation Context Simultaneously. Given our definition of ‘*resolve*’ and *cv*(e), we have the following.

FACT A.52. *Let $\sigma[e[t]]$ be a term such that t is not a value. Then,*

$$\text{cv}(\sigma[e[t]]) = \text{resolve}(\sigma, \text{cv}(e) \cup \text{cv}(t)).$$

The proof proceeds by induction on σ and e , using Facts A.49 and A.51.

A.6.4 Correctness of *cv*.

Definition A.53 (Platform Environment). Γ is a platform environment if for all $x \in \text{dom}(\Gamma)$ we have $x : S^\wedge \{\mathbf{cap}\} \in \Gamma$ for some S .

LEMMA A.54 (INVERSION OF SUBCAPTURING UNDER PLATFORM ENVIRONMENT). *If Γ is a platform environment and $\Gamma \vdash C <: D$, then either $C \subseteq D$ or $\mathbf{cap} \in D$.*

PROOF. By induction on the subcapturing relation. Case (SC-ELEM) trivially holds. Case (SC-SET) holds by repeated IH. In case (SC-VAR), we have $C = \{x\}$ and $x : S^\wedge C' \in \Gamma$. Since Γ is a platform environment, we have $C' = \{\mathbf{cap}\}$, which means that the other premise of (SC-VAR) is $\Gamma \vdash \{\mathbf{cap}\} <: D$. Since Γ is well-formed, $\mathbf{cap} \notin \text{dom}(\Gamma)$, which means that we must have $\mathbf{cap} \in D$. $\square$

LEMMA A.55 (STRENGTHENING OF SUBCAPTURING). *If $\Gamma, \Gamma' \vdash C <: D$ and $C \subseteq \text{dom}(\Gamma)$, then we must have $\Gamma \vdash C <: D$.*

PROOF. First, we consider that if $\mathbf{cap} \in D$, we trivially have the desired goal. If $\mathbf{cap} \notin D$, we proceed by induction on the subcapturing relation. Case (SC-ELEM) trivially holds, and case (SC-SET) holds by repeated IH.

In case (SC-VAR), we have $C = \{x\}$, $x : S^\wedge C' \in \Gamma, \Gamma'$. This implies that $\Gamma = \Gamma_1, x : S^\wedge C', \Gamma_2$ (as $x \notin \text{dom}(\Gamma)$). Since Γ, Γ' is well-formed, we must have $\Gamma_1 \vdash C' \mathbf{wf}$. Since we already know $\mathbf{cap} \notin D$, then we must also have $\mathbf{cap} \notin C'$, which then leads to $C' \subseteq \text{dom}(\Gamma_1)$. This in turn means that by IH and weakening, we have $\Gamma \vdash C' <: D$, and since we also have $x : S^\wedge C' \in \Gamma$, we can conclude by (SC-VAR). $\square$

Then we will need to connect it to subcapturing, because the keys used to open boxes are supercaptures of the capability inside the box. We want the following.

LEMMA A.56. *Let Γ be a platform environment, $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash C_1 <: C_2$. Then $\text{resolve}(\sigma, C_1) \subseteq \text{resolve}(\sigma, C_2)$.*

PROOF. By induction on σ . If σ is empty, we have $\text{resolve}(\sigma, C_1) = C_1$, likewise for C_2 , and we can conclude by Lemma A.54.

Otherwise, $\sigma = \sigma'[\text{let } x = v \text{ in } \]$ and $\Delta = \Delta', x : S_x \wedge D_x$ for some S_x . Let $\theta = \text{resolver}(\sigma)$. We proceed by induction on the subcapturing derivation. Case (SC-ELEM) trivially holds, and case (SC-SET) holds by repeated IH.

In case (SC-VAR), we have $C_1 = \{y\}$ and $y : S_y \wedge D_y \in \Gamma, \Delta$ for some S_y , and $\Gamma, \Delta \vdash D_y <: C_2$. We must have $\Gamma, \Delta' \vdash D_y \text{ wf}$ and so we can strengthen subcapturing to $\Gamma, \Delta' \vdash D_y <: C_2$, which by IH gives us $\text{resolver}(\sigma')(D_y) \subseteq \text{resolver}(\sigma')(C_2)$. By definition, we have $\theta = \text{resolver}(\sigma) = \text{resolver}(\sigma') \circ [x := \text{cv}(v)]$. Since by well-formedness $x \notin D_y$, we now have

$$\theta D_y \subseteq \theta C_2.$$

By Lemma A.38 and Lemma A.47, we must have $\Gamma, \Delta \vdash \text{cv}(v) <: D_y$. Since $\Gamma, \Delta \vdash \text{cv}(v) \text{ wf}$, we can strengthen this to $\Gamma, \Delta \vdash \text{cv}(v) <: D_y$. By outer IH, this gives us $\text{resolver}(\sigma')(\text{cv}(v)) \subseteq \text{resolver}(\sigma')(D_y)$. Since $x \notin \text{cv}(v) \cup D_y$, we have

$$\theta \text{cv}(v) \subseteq \theta D_y,$$

which means we have $\theta \text{cv}(v) \subseteq \theta C_2$ and we can conclude by $\theta \text{cv}(v) = \theta\{x\}$, since

$$\begin{aligned} \theta\{x\} &= (\text{resolver}(\sigma') \circ [x := \text{cv}(v)])(\{x\}) = \text{resolver}(\sigma')(\text{cv}(v)) \\ \theta \text{cv}(v) &= \text{resolver}(\sigma')(\text{cv}(v)) \quad (\text{since } x \notin \text{cv}(v)). \end{aligned}$$

□

A.6.5 Core Lemmas.

LEMMA A.57 (PROGRAM AUTHORITY PRESERVATION). *Let $\Psi[t]$ be a well-typed program such that $\Psi[t] \longrightarrow \Psi[t']$. Then $\text{cv}(t') \subseteq \text{cv}(t)$.*

PROOF. By inspection of the reduction rule used.

Case (APPLY). Then $t = \sigma[e[x y]]$ and $t' = \sigma[e[z := y]s]$. Note that our goal is then

$$\text{resolver}(\sigma)(\text{cv}(e) \cup \text{cv}([z := y]s)) \subseteq \text{resolver}(\sigma)(\text{cv}(e) \cup \text{cv}(x y)).$$

If we have $x \in \text{dom}(\Psi)$, then $\Psi(x) = \lambda(z : U) s$. By definition of platform, the lambda is closed and we have $\text{fv}(s) \subseteq \{z\}$, which in turn means that $\text{cv}([z := y]s) \subseteq \{y\} \subseteq \text{cv}(x y)$. This satisfies our goal.

Otherwise, we have $x \in \text{dom}(\sigma)$ and $\sigma(x) = \lambda(z : U) s$. Since x is bound in σ , we have $\text{resolver}(\sigma)(\text{cv}(\lambda(z : U) s) \cup \{y\}) \subseteq \text{resolver}(\sigma)(\text{cv}(x y))$. Since $\text{cv}([z := y]s) \subseteq \text{cv}(\lambda(z : U) s) \cup \{y\}$, our goal is again satisfied.

Case (TAPPLY). Analogous reasoning.

Case (OPEN). Then $t = \sigma[e[C \multimap x]]$ and $t' = \sigma[e[z]]$. We must have $x \in \text{dom}(\sigma)$ and $\sigma(x) = \Box z$, since all values bound in a platform must be closed and a box form cannot be closed. Since $\Psi[t]$ is a well-typed program, there must exist some Γ, Δ such that Γ is a platform environment and $\vdash \sigma \sim \Gamma, \Delta$.

If $z \in \text{dom}(\sigma)$, then by Lemma A.38 and Lemma A.41 we have $\Gamma, \Delta \vdash z : S_z \wedge C$ for some S_z . By straightforward induction on the typing derivation, we then must have $\Gamma, \Delta \vdash \{z\} <: C$. Then by Lemma A.56, we have $\text{resolver}(\sigma)(\{z\}) \subseteq \text{resolver}(\sigma)(C)$, which lets us conclude by an argument similar to the (APPLY) case.

Otherwise, $z \in \text{dom}(\Psi)$. Here we also have $\Gamma, \Delta \vdash \{z\} <: C$, which implies we must have $z \in C$, so we have $\text{cv}(z) \subseteq \text{cv}(C \multimap x)$ and can conclude by a similar argument as in the (APPLY) case.

Case (RENAME), (LIFT). The lemma is clearly true since these rules only shift subterms of t to create t' . $\square$

LEMMA A.58 (SINGLE-STEP USED CAPABILITY PREDICTION). *Let $\Psi[t] \longrightarrow \Psi[t']$ be a well-typed program such that $\Psi[t] \longrightarrow \Psi[t']$. Then the primitive capabilities used during this reduction are a subset of $\text{cv}(t)$:*

$$\{x \mid x \in \text{used}(\Psi[t] \longrightarrow^* \Psi[t']), x \in \text{dom}(\Psi)\} \subseteq \text{cv}(t).$$

PROOF. By inspection of the reduction rule used.

Case (APPLY). Then $t = \sigma[e[x\ y]]$. If $x \in \text{dom}(\sigma)$, the lemma trivially holds. Otherwise, $x \in \text{dom}(\Psi) \setminus \text{dom}(\sigma)$. From the definition of cv , we have $\{x\} \setminus \text{dom}(\sigma) \subseteq \text{cv}(t)$. Since x is bound in Ψ , we then have $x \in \text{cv}(t)$, which concludes.

Case (TAPPLY). Analogous reasoning.

Case (OPEN), (RENAME), (LIFT). Hold trivially, since no capabilities are used by reducing using these rules. $\square$

THEOREM A.59 (USED CAPABILITY PREDICTION). *Let $\Psi[t] \longrightarrow^* \Psi[t']$, where $\Psi[t]$ is a well-typed program. Then the primitive capabilities used during the reduction are a subset of the authority of t :*

$$\{x \mid x \in \text{used}(\Psi[t] \longrightarrow^* \Psi[t']), x \in \text{dom}(\Psi)\} \subseteq \text{cv}(t).$$

PROOF. By the IH, single-step program trace prediction and authority preservation. $\square$

A.7 Avoidance

Here, we restate Lemma 3.3 and prove it.

LEMMA A.60. *Consider a term $\text{let } x = s \text{ in } t$ in an environment Γ such that $\Gamma \vdash s : R \wedge C_s$ is the most specific typing for s in Γ and $\Gamma, x : R \wedge C_s \vdash t : T$ is the most specific typing for t in the context of the body of the let, namely $\Gamma, x : R \wedge C_s$. Let T' be constructed from T by replacing x with C_s in covariant capture set positions and by replacing x with the empty set in contravariant capture set positions. Then for every type U avoiding x such that $\Gamma, x : S \wedge C_s \vdash T <: U$, we have $\Gamma \vdash T' <: U$.*

PROOF. We will construct a subtyping derivation showing that $T' <: U$. Proceed by structural induction on the subtyping derivation for $T <: U$. Since T' has the same structure as T , most of the subtyping derivation carries over directly except for the subcapturing constraints in (CAPT).

In this case, in covariant positions, whenever we have $C_T <: C_U$ for a capture set C_T from T and a capture set C_U from U , we need to show that $\vdash [x := C_s]C_T <: C_U$. Conversely, in contravariant positions, whenever we have $C_U <: C_T$, we need to show that $C_U <: [x := \{\}]C_T$. For the covariant case, since $x \in C_T$ but not in C_U , by inverting the subcapturing relation $C_T <: C_U$, we obtain $C_s <: C_U$. Hence $[x := C_s]C_T <: C_U$, as desired.

The more difficult case is the contravariant case, when we have $C_U <: C_T$. Here, however, we have that $C_U <: [x := \{\}]C_T$ by structural induction on the subcapturing derivation as x never occurs on the left-hand side of the subcapturing relation as U avoids x . $\square$

B SCOPED CAPABILITY PROOFS

The substitution lemmas leading up to the main correctness theorems need to consider rules (BOUNDARY), (BREAK), (LABEL), (SC-LABEL), (SCOPE). Substitutions clearly preserve subcapturing derived via (SC-LABEL), since the capture sets involved are unaffected. Likewise, substitutions clearly preserve judgments derived via (BREAK), (BOUNDARY), (BREAK) and (SCOPE), based on arguments similar to the ones for rules (FUN), (LET) and (APPLY).

For the progress theorem, we need to add a new canonical forms and lookup inversion lemma.

LEMMA B.1 (SUBTYPING INVERSION: BOUNDARY CAPABILITY TYPE). *If $\Gamma \vdash U <: \text{Break}[S] \wedge C$, then U either is of the form $X \wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash X <: \text{Break}[S]$, or U is of the form $\text{Break}[S'] \wedge C'$ and we have $\Gamma \vdash C' <: C$ and $\Gamma \vdash S <: S'$.*

PROOF. Analogous to the proof of Lemma A.19. □

LEMMA B.2 (CANONICAL FORMS: BOUNDARY CAPABILITY). *If $\Gamma \vdash v : \text{Break}[S] \wedge \{\text{cap}\}$, then $v = l_{S'}$ and $\Gamma \vdash S <: S'$.*

PROOF. Analogous to the proof of Lemma A.34, using Lemma B.1. □

LEMMA B.3 (BOUNDARY CAPABILITY LOOKUP INVERSION). *If $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash x : \text{Break}[S]$ and $\sigma(x) = l_{S'}$, and $\Gamma, \Delta \vdash S <: S'$.*

PROOF. A corollary of Lemma A.38 and Lemma B.2. □

THEOREM B.4 (PRESERVATION). *Let $\Gamma \vdash \sigma \sim \Delta$ and $\Gamma, \Delta \vdash t : T$, where $\sigma[t]$ is a proper program. Then $\sigma[t] \longrightarrow \sigma[t']$ implies that $\Gamma, \Delta \vdash t' : T$ and that $\sigma[t']$ is a proper program.*

PROOF. We proceed by inspecting the rule used to reduce $\sigma[t]$.

Case (APPLY). Then we have $t = e[x y]$ and $\sigma(x) = \lambda(z : U) s$ and $t' = e[z := y]s$.

By Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash x y : Q$. The typing derivation of $x y$ must start with an arbitrary number of (SUB) rules, followed by (APP). We proceed by induction on the number of (SUB) rules. In both base and inductive cases, we can only assume that $\Gamma, \Delta \vdash x y : Q'$ for some Q' such that $\Gamma, \Delta \vdash Q' <: Q$.

Inductive Case. $\Gamma, \Delta \vdash x y : Q'$ is derived by (SUB), so we also have some Q'' such that $\Gamma, \Delta \vdash x y : Q''$ and $\Gamma, \Delta \vdash Q'' <: Q'$. We have $\Gamma, \Delta \vdash Q'' <: Q$ by (TRANS), so we can conclude by using the inductive hypothesis on $\Gamma, \Delta \vdash x y : Q''$.

Base Case. $\Gamma, \Delta \vdash x y : Q'$ is derived by (APP), so for some Q'' we have $\Gamma, \Delta \vdash x : \forall(z : U') Q''$ and $\Gamma, \Delta \vdash y : U'$ and $Q' = [z := y]Q''$.

By Lemma A.39, we have $\Gamma, \Delta, z : U' \vdash s : Q''$. By Lemma A.29, we have $\Gamma, \Delta \vdash [z := y]s : [z := y]Q''$, and since $Q' = [z := y]Q''$, by (SUB) we have $\Gamma, \Delta \vdash [z := y]s : Q$.

By Lemma A.5, we have $\Gamma, \Delta \vdash e[[z := y]s] : T$, our first goal.

Our second goal is showing that $\sigma[e[t']]$ is a proper program. We have

$$\text{cv}(\sigma[e[t]]) = \text{resolver}(\sigma)(\text{cv}(e) \cup \text{cv}(t))$$

$$\text{cv}(\sigma[e[t']]) = \text{resolver}(\sigma)(\text{cv}(e) \cup \text{cv}(t')).$$

Since $\text{resolver}(\sigma)(\text{cv}(x)) = \text{resolver}(\sigma)(\text{cv}(\lambda(z : U) s))$, we also have

$$\text{resolver}(\sigma)(\text{cv}(t')) \subseteq \text{resolver}(\sigma)(\text{cv}(t)).$$

In other words, the cv of the program does not increase, which means we clearly preserve the first criterion of a proper program since the evaluation context remains the same. Similarly, since all the scope forms were part of e , we also preserve the second criterion, which lets us conclude.

Case (TAPPLY). As above.

Case (OPEN). Then we have $t = e[C \multimap x]$ and $\sigma(x) = \Box y$ and $t' = e[y]$.

The argument is nearly the same as for rule (APPLY), with a small difference when showing that the result is a proper program.

In the base induction case, we have $\Gamma, \Delta \vdash x : \Box S^{\wedge} C$. By Lemma A.38 and Lemma A.41, we also have $\Gamma, \Delta \vdash y : S^{\wedge} C$ for some S . Then by a straightforward induction on the typing derivation, we must have $\Gamma, \Delta \vdash \{y\} <: C$. Then by Lemma A.56, we have $\text{resolver}(\sigma)(\{y\}) \subseteq \text{resolver}(\sigma)(C)$. This lets us know that for every $x \in \text{resolver}(\sigma)(\text{cv}(t'))$, we also have $x \in \text{resolver}(\sigma)(\text{cv}(t))$, which lets us carry out the argument from the case for rule (APPLY).

Case (RENAME). Then $t = e[\text{let } x = y \text{ in } s]$ and $t' = e[[x := y]s]$.

Again, by Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \text{let } x = y \text{ in } s : Q$. As in the (APPLY) case, we proceed by induction, only working with a Q' such that $Q' <: Q$. The inductive case remains the same.

Base Case. (LET) was used to derive that $\Gamma, \Delta \vdash \text{let } x = y \text{ in } s : Q'$. The premises are $\Gamma, \Delta \vdash y : U$ and $\Gamma, \Delta, x : U \vdash s : Q'$ and $x \notin \text{fv}(Q')$. By Lemma A.29, we have $\Gamma, \Delta \vdash [x := y]s : [x := y]Q'$ and $x \notin \text{fv}(Q')$, $[x := y]Q' = Q'$. We conclude by reasoning similar to the (APPLY) case: first goal holds by (SUB) and Lemma A.5, second goal holds since the cv of the program does not increase and e remains the same.

Case (LIFT). Then we have $t = e[\text{let } x = v \text{ in } s]$ and $t' = \text{let } x = v \text{ in } e[s]$.

Again, by Lemma A.4 for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \text{let } x = v \text{ in } s : Q$. As in the (APPLY) case, we proceed by induction, only working with a Q' such that $Q' <: Q$. The inductive case remains the same.

Base Case. (LET) was used to derive that $\Gamma, \Delta \vdash \text{let } x = v \text{ in } s : Q'$. The premises are $\Gamma, \Delta \vdash v : U$ and $\Gamma, \Delta, x : U \vdash s : Q'$ and $x \notin \text{fv}(Q')$.

By weakening of typing, we also have $\Gamma, \Delta, x : U \vdash e : Q \Rightarrow T$. Then by (SUB) and Lemma A.5, we have $\Gamma, \Delta, x : U \vdash e[s] : T$. Since $\Gamma, \Delta \vdash T \text{ wf}$, by Barendregt $x \notin \text{fv}(T)$, so by (LET) we have $\Gamma, \Delta \vdash \text{let } x = v \text{ in } e[s] : T$. We conclude by reasoning similar to the (APPLY) case: first goal was shown, second goal holds since the cv of the program does not increase and e remains the same.

Case (ENTER). Then $t = e[\text{boundary}[S] x \Rightarrow t]$ and $t' = \text{let } x = l_S \text{ in } e[\text{scope}_x t]$.

By Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \text{boundary}[S] x \Rightarrow t : Q$. As in the (APPLY) case, we proceed by induction, only working with a Q' such that $Q' <: Q$. The inductive case remains the same.

Base Case. $\Gamma, \Delta \vdash \text{boundary}[S] x \Rightarrow t : Q'$ was derived via (BOUNDARY), so we have $Q' = S$ and $\Gamma, \Delta, x : \text{Break}[S] \vdash t : S$ and $x \notin \text{fv}(S)$. This means that we can derive $\Gamma, \Delta \vdash \text{let } x = l_S \text{ in } e[\text{scope}_{l_S} t] : S$ via (SCOPE), (SUB) and Lemma A.5, and finally (LET).

Similarly to the case for (APPLY), we have $\text{resolver}(\zeta)(t') \subseteq \text{resolver}(\zeta)(t) \cup \{x\}$. By Barendregt, we must have $x \notin \text{resolver}(\zeta)(e)$, which means that $\sigma[e[t']]$ remains a proper program.

Case (BREAK). Then $t = e_1[\text{scope}_{l_S} e_2[x y]]$ and $\sigma(x) = l_S$ and $t' = e_1[y]$.

By Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e_1[\text{scope}_{l_S} e_2] : Q \Rightarrow T$ and $\Gamma, \Delta \vdash x y : Q$. As in the (APPLY) case, we proceed by induction, only working with a Q' such that $Q' <: Q$. The inductive case remains the same.

Base Case. $\Gamma, \Delta \vdash xy : Q'$ was derived via (INVOKE), so for some S' we have $\Gamma, \Delta \vdash x : \text{Break}[S']$ and $\Gamma, \Delta \vdash y : S'$. By Lemma B.3, we have $\Gamma, \Delta \vdash S' <: S$. We now use Lemma A.4 once again, this time on e_1 : for some P , we have $\Gamma, \Delta \vdash e_1 : P \Rightarrow T$ and $\Gamma, \Delta \vdash \text{scope}_{l_S} e_2[xy] : P$. We proceed by induction yet again just like in the (APPLY) case, only working with a P' such that $P' <: P$. The inductive case remains unchanged.

Base Case. $\Gamma, \Delta \vdash \text{scope}_{l_S} e_2[xy] : P'$ was derived via (SCOPE), which means that $P' = S$, which in turn gives us $\Gamma, \Delta \vdash S <: P$, and by transitivity $\Gamma, \Delta \vdash S' <: P$, which means that we can conclude that $\Gamma, \Delta \vdash e_1[y] : T$ by (SUB) and Lemma A.5.

We have $\Gamma, \Delta \vdash y : S$ and accordingly $\text{resolver}(\sigma)(y) = \{\}$. This implies that if we have $x \in \text{resolver}(\sigma)(\text{cv}(e_1[y]))$, then $x \in \text{resolver}(\sigma)(e_1)$. Since e_1 is only one part of the proper program $\sigma[e_1[\text{scope}_x e_2]]$, clearly $\sigma[e_1[y]]$ remains well-scoped.

Case (LEAVE). Then $t = e[\text{scope}_{l_S} a]$ and $t' = e[a]$.

By Lemma A.4, for some Q we have $\Gamma, \Delta \vdash e : Q \Rightarrow T$ and $\Gamma, \Delta \vdash \text{scope}_{l_S} a : Q$. As in the (APPLY) case, we proceed by induction, only working with a Q' such that $Q' <: Q$. The inductive case remains the same.

Base Case. $\Gamma, \Delta \vdash \text{scope}_{l_S} a : Q'$ was derived via (SCOPE), which means that $Q' = S$ and that $\Gamma, \Delta \vdash a : S$, which lets us conclude that $\Gamma, \Delta \vdash e[a] : T$ by (SUB) and Lemma A.5.

Since $\Gamma, \Delta \vdash a : S$, we have $\text{cv}(\sigma[a]) = \{\}$. Since we must have had $l \notin \text{cv}(\sigma[e])$, $\sigma[e[a]]$ remains a proper program. $\square$

THEOREM B.5 (PROGRESS). *Let $\vdash \sigma[e[t]] : T$, where $\sigma[e[t]]$ is a proper configuration and a proper program. Then either $e[t] = a$ for some a or $\sigma[e[t]] \longrightarrow \sigma[t']$ for some t' .*

PROOF. Since $\sigma[e[t]]$ is well-typed in the empty environment, there clearly must be some Δ such that $\vdash \sigma \sim \Delta$ and $\Delta \vdash e[t] : T$. By Lemma A.4, we have that $\Delta \vdash t : P$ for some P . We proceed by induction on the structure of this derivation.

Case (VAR). Then $t = x$.

If e is non-empty, $e[x] = e'[\text{let } y = x \text{ in } t']$ and we can step by (RENAME); otherwise, we conclude with $a = x$.

Case (ABS), (TABS), (BOX). Then $t = v$.

If e is non-empty, $e[v] = e'[\text{let } x = v \text{ in } t']$ and we can step by (LIFT); otherwise, we conclude with $a = v$.

Case (APP). Then $t = xy$ and $\Delta \vdash x : (\forall(z : U) T_0)^\wedge C$ and $\Delta \vdash y : U$.

By Lemmas A.45 and A.34, $\sigma(x) = \lambda(z : U') t'$, which means we can step by (APPLY).

Case (TAPP). Then $t = x[S]$ and $\Delta \vdash x : (\forall[Z <: S] T_0)^\wedge C$.

By Lemmas A.45 and A.35, $\sigma(x) = \lambda[z <: S'] t'$, which means we can step by (TAPPLY).

Case (UNBOX). Then $t = C \circ x$ and $\Delta \vdash x : \Box S^\wedge C$. By Lemmas A.45 and A.36, $\sigma(x) = \Box y$, which means we can step by (OPEN).

Case (LET). Then $t = \text{let } x = u \text{ in } t'$ and we proceed by IH on u , with $e[\text{let } x = [] \text{ in } t']$ as the evaluation context.

Case (SUB). By IH.

Case (BOUNDARY). Then $t = \text{boundary}[S] x \Rightarrow t'$ and $\Delta, x : \text{Break}[S] \vdash t' : S$, and we can step by (ENTER).

Case (SCOPE). Then $t = \text{scope}_{l_S} t'$ and we proceed by IH on t , with $e[\text{scope}_{l_S} []]$ as the evaluation context.

Case (INVOKE). Then $t = xy$ and $\Delta \vdash x : \text{Break}[S]$ and $\Delta \vdash y : S$.

By Lemmas A.45 and B.2, we have $\sigma(x) = l_S$; since $\sigma[e[t]]$ is a proper program, e must be of the form $e_1[\text{scope}_{l_S} e_2]$. Then we can step by (BREAK). $\square$

B.1 Predicting Used Capabilities

LEMMA B.6 (PROGRAM AUTHORITY PRESERVATION). *Let $t \longrightarrow t'$, where $\vdash t : T$. Then,*

$$cv(t') \subseteq cv(t) \cup \text{gained}(t \longrightarrow t').$$

PROOF. We start by inspecting the evaluation rule used.

Case (APPLY). Then $t = \sigma[e[x y]]$ and $u = \sigma[e[[z := y]s]]$, and $\text{gained}(t \longrightarrow u) = \{\}$. Then our goal is

$$\text{resolver}(\sigma)(cv(e) \cup cv([z := y]s)) \subseteq \text{resolver}(\sigma)(cv(e) \cup cv(xy)).$$

We clearly must have $x \in \text{dom}(\sigma)$ and $\sigma(x) = \lambda(z : U) s$. Since x is bound in σ , we have $\text{resolver}(\sigma)(cv(\lambda(z : U) s) \cup \{y\}) \subseteq \text{resolver}(\sigma)(cv(xy))$. Since $cv([z := y]s) \subseteq cv(\lambda(z : U) s) \cup \{y\}$, our goal is again satisfied.

Case (TAPPLY). Analogous reasoning.

Case (OPEN). Then $t = \sigma[e[C \multimap x]]$ and $u = \sigma[e[z]]$ and $\text{gained}(t \longrightarrow u) = \{\}$. We must have $x \in \text{dom}(\sigma)$ and $z \in \text{dom}(\sigma)$ and $\sigma(x) = \Box z$. Since t is well-typed, we clearly must have $\vdash \sigma \sim \Delta$ for some Δ .

By Lemma A.38 and Lemma A.41, we have $\Gamma, \Delta \vdash z : S_z \wedge C$ for some S_z . By a straightforward induction on the typing derivation, we must then have $\Gamma, \Delta \vdash \{z\} <: C$. Then by Lemma A.56, we have $\text{resolver}(\sigma)(\{z\}) \subseteq \text{resolver}(\sigma)(C)$, which lets us conclude by an argument similar to the (APPLY) case.

Case (RENAME), (LIFT). The lemma is clearly true since these rules only shift subterms of t to create u .

Case (ENTER). Then $t = \sigma[e[\text{boundary}[S]x \Rightarrow u]]$ and $u = \text{let } x = l_S \text{ in } \sigma[e[\text{scope}_{l_S} u]]$ and $\text{gained}(t \longrightarrow u) = \{x\}$. We have both

$$cv(t) = \text{resolver}(\sigma)(cv(e) \cup (cv(u) \setminus x))$$

$$cv(u) = (\text{resolver}(\sigma) \circ [x := \{l\}]) (cv(e) \cup cv(u)).$$

We have $x \notin cv(e)$ by Barendregt, which tells us that

$$cv(u) = \text{resolver}(\sigma)(cv(e) \cup [x := \{l\}] cv(u)).$$

Then we have $cv(u) \subseteq \text{resolver}(\sigma)(cv(e) \cup (cv(u) \setminus x)) \cup \{l\}$, which concludes.

Case (BREAK). Then $t = \sigma[e_1[\text{scope}_x e_2[x y]]]$ and $u = \sigma[e_1[y]]$ and $\sigma(x) = \cdot$. We have

$$cv(t) = \text{resolver}(\sigma)(cv(e_1) \cup cv(e_2) \cup cv(xy))$$

$$cv(u) = \text{resolver}(\sigma)(cv(e_1) \cup cv(y)).$$

Since $cv(y) \subseteq cv(xy)$, we have $cv(t) \subseteq cv(u)$ and we can conclude.

Case (LEAVE). Then $t = \sigma[e[\text{scope}_{l_S} a]]$ and $u = \sigma[e[a]]$.

We have $cv(t) = cv(u) = \text{resolver}(\sigma)(cv(e) \cup cv(a))$ and we can conclude. $\square$

LEMMA B.7 (SINGLE-STEP USED CAPABILITY PREDICTION). *Let $t \longrightarrow u$, where $\vdash t : T$. Then the primitive capabilities used during the evaluation are within the authority of t :*

$$\text{used}(t \longrightarrow u) \subseteq cv(t).$$

PROOF. By inspection of the reduction rule used.

Case (BREAK). Then $t = \sigma[e_1[\text{scope}_{l_S} e_2[x y]]]$ and $u = \sigma[e_1[y]]$ and $\sigma(x) = l_S$ and $\text{used}(t \longrightarrow u) = \{l\}$. We have

$$cv(t) = \text{resolver}(\sigma)(cv(e_1) \cup cv(e_2) \cup cv(xy)).$$

Since $\sigma(x) = l_S$, $\text{resolver}(\sigma)(x) = l$ and so we have $l \in cv(t)$.

Case(ENTER), (LEAVE), (APPLY), (TAPPLY), (OPEN), (RENAME), (LIFT). The lemma holds trivially, since no capabilities are used by reducing using these rules. $\square$

THEOREM B.8 (USED CAPABILITY PREDICTION). *Let $t \longrightarrow^* t'$, where $\vdash t : T$. Then the primitive capabilities used during the evaluation are within the authority of t :*

$$\text{used}(t \longrightarrow^* t') \subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow^* t').$$

PROOF. We begin by induction on the number of evaluation steps. The theorem is trivially true for the base case of 0 steps. In the inductive case, we have $t \longrightarrow t'' \longrightarrow^* t'$.

By Lemma B.6, we have

$$\text{cv}(t') \subseteq \text{cv}(t) \cup \text{gained}(t \longrightarrow t'') \subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow t'').$$

By the IH, we have $\text{used}(t'' \longrightarrow^* t') \subseteq \text{cv}(t') \cup \text{created}(t'' \longrightarrow^* t')$, which gives us

$$\begin{aligned} \text{used}(t'' \longrightarrow^* t') &\subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow t'') \cup \text{created}(t'' \longrightarrow^* t') \\ &\subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow^* t'). \end{aligned}$$

By Lemma B.7, we have $\text{used}(t \longrightarrow t'') \subseteq \text{cv}(t)$.

Then $\text{used}(t \longrightarrow t') \subseteq \text{cv}(t) \cup \text{created}(t \longrightarrow^* t')$, which concludes. $\square$

ACKNOWLEDGMENTS

We thank Jonathan Aldrich, Vikraman Choudhury, and Neel Krishnaswami for their input in discussions about this research. We thank the anonymous reviewers of previous versions of this article for their comments and suggestions. In particular, we paraphrased with permission one reviewer's summary of our design decisions in the conclusion. We thank Yichen Xu and Joseph Fourment for feedback and validation of the metatheory, notably through its mechanization.

REFERENCES

- Nada Amin, Samuel Grütter, Martin Odersky, Tiark Rompf, and Sandro Stucki. 2016. The essence of dependent object types. In *A List of Successes That Can Change the World*. Springer, 249–272. https://doi.org/10.1007/978-3-319-30936-1_14
- Yuyan Bao, Guannan Wei, Oliver Bracevac, Yuxuan Jiang, Qiyang He, and Tiark Rompf. 2021. Reachability types: Tracking aliasing and separation in higher-order functional programs. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–32. <https://doi.org/10.1145/3485516>
- Erik Barendsen and Sjaak Smetsers. 1996. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science* 6, 6 (Dec. 1996), 579–612. <https://doi.org/10.1017/S0960129500070109>
- Dariusz Biernacki, Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2020. Binders by day, labels by night: Effect instances via lexically scoped handlers. In *Proceedings of the Symposium on Principles of Programming Languages*. ACM, New York, NY.
- Corrado Böhm and Alessandro Berarducci. 1985. Automatic synthesis of typed λ -programs on term algebras. *Theoretical Computer Science* 39 (1985), 135–154. [https://doi.org/10.1016/0304-3975\(85\)90135-5](https://doi.org/10.1016/0304-3975(85)90135-5)
- Aleksander Boruch-Gruszecki, Jonathan Immanuel Brachthäuser, Edward Lee, Ondřej Lhoták, and Martin Odersky. 2021. Tracking captured variables in types. *arXiv:2105.11896 [cs]* (2021).
- John Boyland, James Noble, and William Retert. 2001. Capabilities for sharing. In *ECOOP 2001—Object-Oriented Programming*, Jørgen Lindskov Knudsen (Ed.). Springer, Berlin, Germany, 2–27.
- Gilad Bracha, Peter von der Ahé, Vassili Bykov, Yaron Kishai, William Maddox, and Eliot Miranda. 2010. Modules as objects in Newspeak. In *ECOOP 2010—Object-Oriented Programming*. Lecture Notes in Computer Science, Vol. 6183. Springer, 405–428. https://doi.org/10.1007/978-3-642-14107-2_20
- Jonathan Immanuel Brachthäuser, Philipp Schuster, Edward Lee, and Aleksander Boruch-Gruszecki. 2022. Effects, capabilities, and boxes: From scope-based reasoning to type-based reasoning and back. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), Article 76, 30 pages. <https://doi.org/10.1145/3527320>
- Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020a. Effects as capabilities: Effect handlers and lightweight effect polymorphism. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), Article 126, 30 pages. <https://doi.org/10.1145/3428194>

- Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020b. Effekt: Capability-passing style for type- and effect-safe, extensible effect handlers in Scala. *Journal of Functional Programming* 30 (2020), e8. <https://doi.org/10.1017/S0956796820000027>
- Vikraman Choudhury and Neel Krishnaswami. 2020. Recovering purity with comonads and capabilities. *Proceedings of the ACM on Programming Languages* 4, ICFP (Aug. 2020), Article 111, 28 pages. <https://doi.org/10.1145/3408993>
- David G. Clarke, John M. Potter, and James Noble. 1998. Ownership types for flexible alias protection. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '98)*. ACM, New York, NY, 48–64. <https://doi.org/10.1145/286936.286947>
- William R. Cook. 2009. On understanding data abstraction, revisited. In *Proceedings of the 24th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Language, and Applications (OOPSLA'09)*. ACM, New York, NY, 557–572.
- Aaron Craig, Alex Potanin, Lindsay Groves, and Jonathan Aldrich. 2018. Capabilities: Effects for free. In *Formal Methods and Software Engineering*. Lecture Notes in Computer Science, Vol. 11232. Springer, 231–247. https://doi.org/10.1007/978-3-030-02450-5_14
- Karl Crary, David Walker, and Greg Morrisett. 1999. Typed memory management in a calculus of capabilities. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'99)*. ACM, New York, NY, 262–275. <https://doi.org/10.1145/292540.292564>
- Sophia Drossopoulou, James Noble, Mark S. Miller, and Toby Murray. 2016. Permission and authority revisited towards a formalisation. In *Proceedings of the 18th Workshop on Formal Techniques for Java-Like Programs (FTJP'16)*. ACM, New York, NY, 1–6. <https://doi.org/10.1145/2955811.2955821>
- Ismael Figueroa, Nicolas Tabareau, and Éric Tanter. 2016. Effect capabilities for Haskell: Taming effect interference in monadic programming. *Science of Computer Programming* 119 (April 2016), 3–30. <https://doi.org/10.1016/j.scico.2015.11.010>
- Joseph Fourment and Yichen Xu. 2023. *A Mechanized Theory of the Box Calculus*. Technical Report. Infoscience. <http://infoscience.epfl.ch/record/302949>
- Colin S. Gordon. 2020. Designing with static capabilities and effects: Use, mention, and invariants (Pearl). In *Designing with Static Capabilities and Effects: Use, Mention, and Invariants*. Leibniz International Proceedings in Informatics, Vol. 166, Robert Hirschfeld and Tobias Pape (Eds.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Article 10, 25 pages. <https://doi.org/10.4230/LIPIcs.ECOOP.2020.10>
- Dan Grossman, Greg Morrisett, Trevor Jim, Michael Hicks, Yanling Wang, and James Cheney. 2002. Region-based memory management in Cyclone. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI'02)*. ACM, New York, NY, 282–293. <https://doi.org/10.1145/512529.512563>
- John Hannan. 1998. A type-based escape analysis for functional languages. *Journal of Functional Programming* 8, 3 (May 1998), 239–273.
- John Hatcliff and Olivier Danvy. 1994. A generic account of continuation-passing styles. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'94)*. ACM, New York, NY, 458–471. <https://doi.org/10.1145/174675.178053>
- John Launchbury and Amr Sabry. 1997. Monadic state: Axiomatization and type safety. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP'97)*. ACM, New York, NY, 227–238. <https://doi.org/10.1145/258948.258970>
- Daan Leijen. 2014. Koka: Programming with row polymorphic effect types. *Electronic Proceedings in Theoretical Computer Science* 153 (June 2014), 100–126. <https://doi.org/10.4204/EPTCS.153.8> arXiv:1406.2061
- Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. In *Proceedings of the Symposium on Principles of Programming Languages*. ACM, New York, NY, 486–499. <https://doi.org/10.1145/3009837.3009872>
- Sam Lindley, Conor McBride, and Craig McLaughlin. 2017. Do be do be do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL'17)*. ACM, New York, NY, 500–514. <https://doi.org/10.1145/3009837.3009897>
- Fengyun Liu. 2016. *A Study of Capability-Based Effect Systems*. Master's Thesis. Infoscience. infoscience.epfl.ch/record/219173
- J. M. Lucassen and D. K. Gifford. 1988. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '88)*. ACM, New York, NY, 47–57. <https://doi.org/10.1145/73560.73564>
- Daniel Marino and Todd D. Millstein. 2009. A generic type-and-effect system. In *Proceedings of the 2009 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation (TLDI'09)*. ACM, New York, NY, 39–50. <https://doi.org/10.1145/1481861.1481868>
- Darya Melicher. 2020. *Controlling Module Authority Using Programming Language Design*. Ph.D. Dissertation. Carnegie Mellon University.

- Darya Melicher, Yangqingwei Shi, Alex Potanin, and Jonathan Aldrich. 2017. A capability-based module system for authority control. In *Proceedings of the 31st European Conference on Object-Oriented Programming (ECOOP'17)*.
- Mark Samuel Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph.D. Dissertation. Johns Hopkins University.
- Aleksandar Nanevski, Frank Pfenning, and Brigitte Pientka. 2008. Contextual modal type theory. *ACM Transactions on Computational Logic* 9, 3 (June 2008), Article 23, 49 pages. <https://doi.org/10.1145/1352582.1352591>
- James Noble, Jan Vitek, and John Potter. 1998. Flexible alias protection. In *ECOOP'98—Object-Oriented Programming*. Lecture Notes in Computer Science, Vol. 1445. Springer, 158–185. <https://doi.org/10.1007/BFb0054091>
- Martin Odersky, Olivier Blanvillain, Fengyun Liu, Aggelos Biboudis, Heather Miller, and Sandro Stucki. 2018. Simplytly: Foundations and applications of implicit function types. *Proceedings of the ACM on Programming Languages* 2, POPL (Dec. 2008), Article 42, 29 pages. <https://doi.org/10.1145/3158130>
- Martin Odersky, Aleksander Boruch-Gruszecki, Jonathan Immanuel Brachthäuser, Edward Lee, and Ondřej Lhoták. 2021. Safer exceptions for Scala. In *Proceedings of the Scala Symposium*. <https://dl.acm.org/doi/10.1145/3486610.3486893>
- Martin Odersky and Guillaume Martres. 2020. Extension Methods: Scala 3 Language Reference Page. Retrieved September 19, 2023 from <https://dotty.epfl.ch/docs/reference/contextual/extension-methods.html>
- Leo Osvald, Grégory M. Essertel, Xilun Wu, Lilliam I. González Alayón, and Tiark Rompf. 2016. Gentrification gone too far? Affordable 2nd-class values for fun and (co-)effect. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'16)*. ACM, New York, NY, 234–251. <https://doi.org/10.1145/2983990.2984009>
- Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2014. Coeffects: A calculus of context-dependent computation. In *Proceedings of the International Conference on Functional Programming (Gothenburg, Sweden)*. ACM, New York, NY, 123–135. <https://doi.org/10.1145/2628136.2628160>
- Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press, Cambridge, MA.
- Tiark Rompf and Nada Amin. 2016. Type soundness for dependent object types (DOT). In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'16)*. ACM, New York, NY, 624–641. <https://doi.org/10.1145/2983990.2984008>
- Lukas Rytz, Martin Odersky, and Philipp Haller. 2012. Lightweight polymorphic effects. In *ECOOP 2012—Object-Oriented Programming*. Lecture Notes in Computer Science, Vol. 7313. Springer, 258–282. https://doi.org/10.1007/978-3-642-31057-7_13
- Amr Sabry and Matthias Felleisen. 1993. Reasoning about programs in continuation-passing style. In *LISP and Symbolic Computation* 6 (1993), 289–360.
- Scala. 2022a. Scala 3 API: scala.util.boundary. Retrieved September 19, 2023 from <https://www.scala-lang.org/api/3.3.0/scala/util/boundary/protect/T1/textdollar.html>
- Scala. 2022b. Scala 3: Capture Checking. Retrieved September 19, 2023 from <https://dotty.epfl.ch/docs/reference/experimental/cc.html>
- Scala. 2022c. The Scala 3 Compiler, Also Known as Dotty. Retrieved September 19, 2023 from <https://dotty.epfl.ch>
- Gabriel Scherer and Jan Hoffmann. 2013. Tracking data-flow with open closure types. In *Proceedings of the International Conference on Logic for Programming Artificial Intelligence and Reasoning*. 710–726. https://doi.org/10.1007/978-3-319-30936-1_14
- Jeremy G. Siek, Michael M. Vitousek, and Jonathan D. Turner. 2012. Effects for funargs. *CoRR abs/1201.0023* (2012). <http://arxiv.org/abs/1201.0023>
- Mads Tofte and Jean-Pierre Talpin. 1997. Region-based memory management. *Information and Computation* 132, 2 (Feb. 1997), 109–176. <https://doi.org/10.1006/inco.1996.2613>
- Philip Wadler. 1990. Linear types can change the world! In *Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2, 2.3 Working Conference on Programming Concepts and Methods*, Manfred Broy and Cliff B. Jones (Eds.). North-Holland, 561.
- Yichen Xu and Martin Odersky. 2023. *Formalizing Box Inference for Capture Calculus*. Technical Report. EPFL.
- Yizhou Zhang and Andrew C. Myers. 2019. Abstraction-safe effect handlers via tunneling. *Proceedings of the ACM on Programming Languages* 3, POPL (Jan. 2019), Article 5, 29 pages. <https://doi.org/10.1145/3290318>

Received 23 October 2022; revised 25 June 2023; accepted 1 August 2023